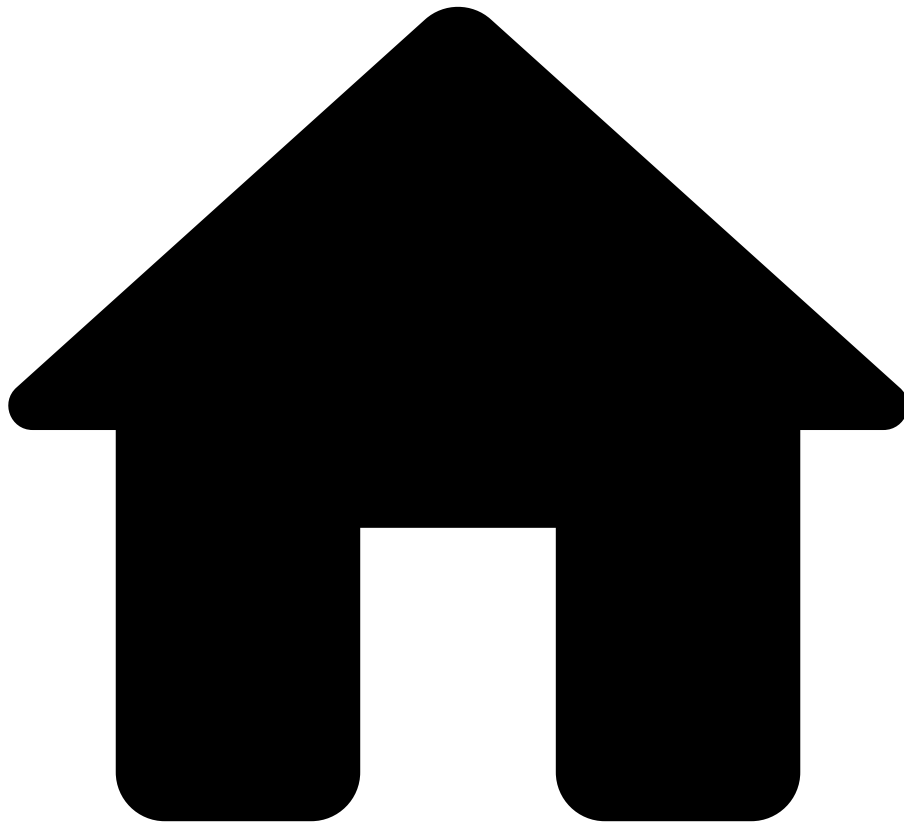


Forward NQE Docs - Part 8

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- address

On this page

address

```
address(subnet: IpSubnet) : IpAddress
```

Returns the IP address of a subnet. For example,

```
address(ipSubnet("1.2.3.4/24")) is ipAddress("1.2.3.4") .
```

See also[â€‹](#)

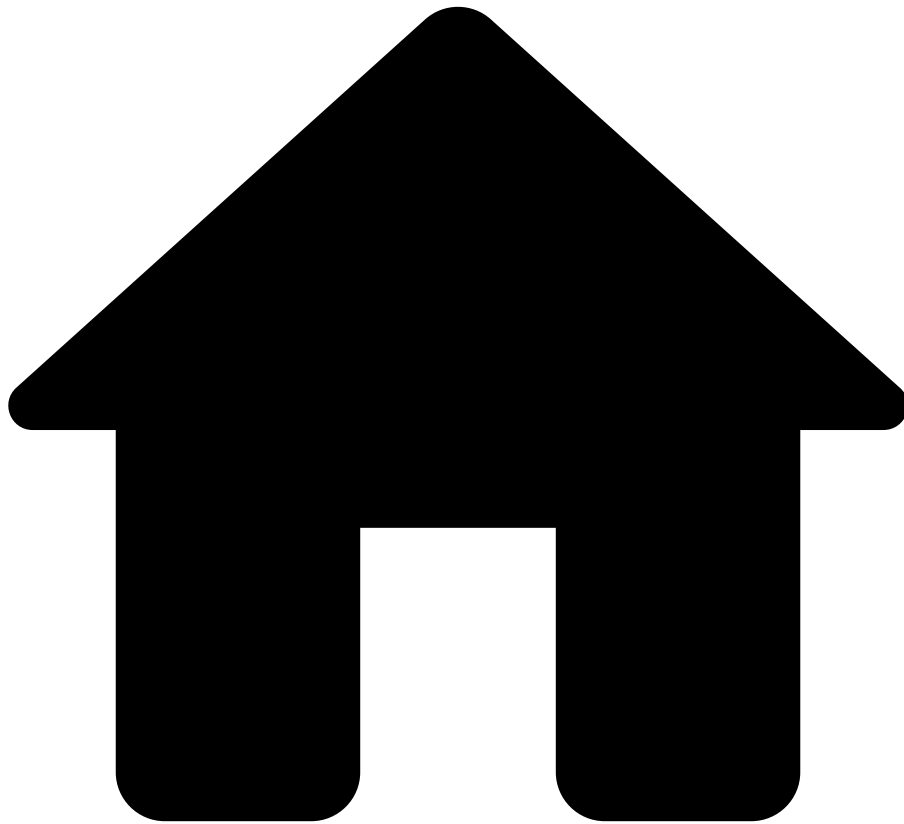
Types[â€‹](#)

- [IpAddress](#)
- [IpSubnet](#)

Functions[â€‹](#)

- [ipAddress](#)
- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- all

On this page

all

```
all(booleans: Bag<Bool>) : Bool
```

Returns `true` if all elements of the given collection `booleans` are `true`.

- If the `booleans` collection is empty, returns `true`.
- A `null` element is considered `false`.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

Examples

```
emptyList =
  foreach x in fromTo(1, 0)
  select true;

[
  { allWithOnlyTrues: all([true, true]), /* ==> true */
    allWithOnlyFalses: all([false, false]), /* ==> false */
    allWithSomeTrues: all([true, false, true]), /* ==> false */
  }
]
```

The following example query finds for each device if *all* password-policy config satisfies long password length:

```
pattern = ``
  password-policy minimum-length {minLength:number}
``;

LONG_PASSWORD_LENGTH = 15;

foreach device in network.devices
let config = device.files.config
select {
  Device: device.name,
  OnlyLongPassword: if hasBlockMatch(config, pattern)
    then all(foreach match in
      blockMatches(config, pattern)
```

```
select match.data.minLength >=
LONG_PASSWORD_LENGTH)
else false
}
```

See also[â€‹](#)

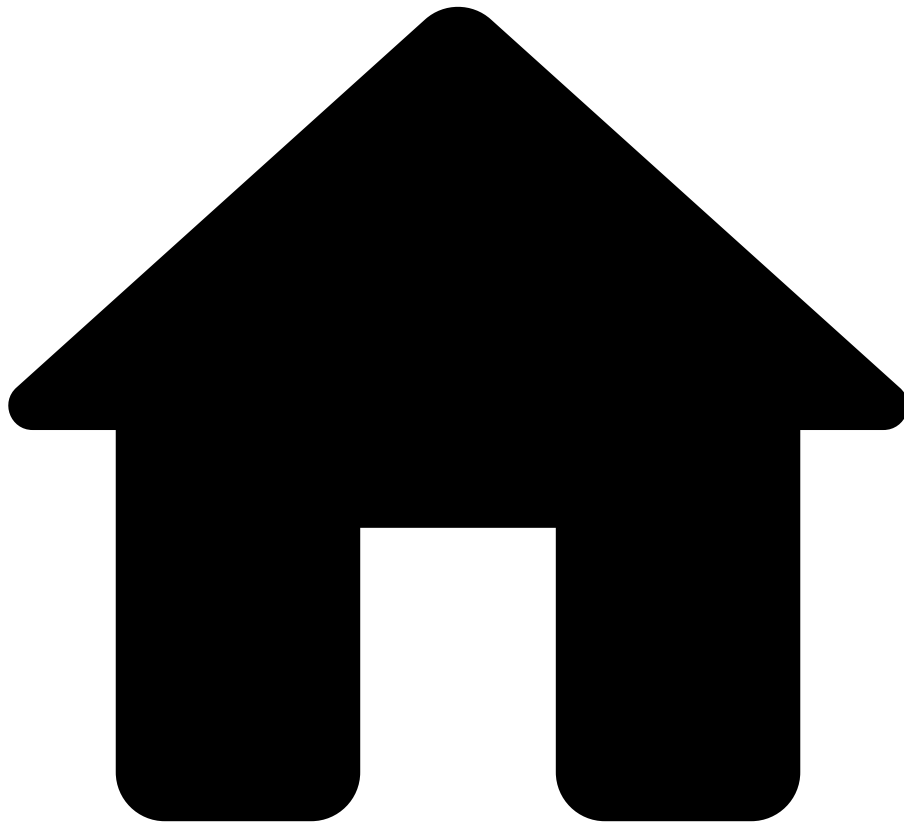
Types[â€‹](#)

- [Bool](#)
- [Collections](#)

Functions[â€‹](#)

- [any](#)
- [min](#)
- [max](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- any

On this page

any

```
any(booleans: Bag<Bool>) : Bool
```

Returns `true` if any element of the given collection `booleans` is `true`.

- If the `booleans` collection is empty, returns `false`.
- A `null` element is considered `false`.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

Examples

```
emptyList =
  foreach x in fromTo(1, 0)
  select true;

[
  { anyWithOnlyTrues: any([true, true]), /* ==> true */
    anyWithOnlyFalses: any([false, false]), /* ==> false */
    anyWithSomeTrues: any([false, false, true]), /* ==> true */
  }
]
```

The following example query finds for each device if any password-policy config satisfies long password length:

```
pattern = ``
password-policy minimum-length {minLength:number}
``;

LONG_PASSWORD_LENGTH = 15;

foreach device in network.devices
let config = device.files.config
select {
  Device: device.name,
  HasLongPassword: any(foreach match in blockMatches(config,
pattern)
                        select match.data.minLength >=
```

```
LONG_PASSWORD_LENGTH)  
}
```

See also[â€‹](#)

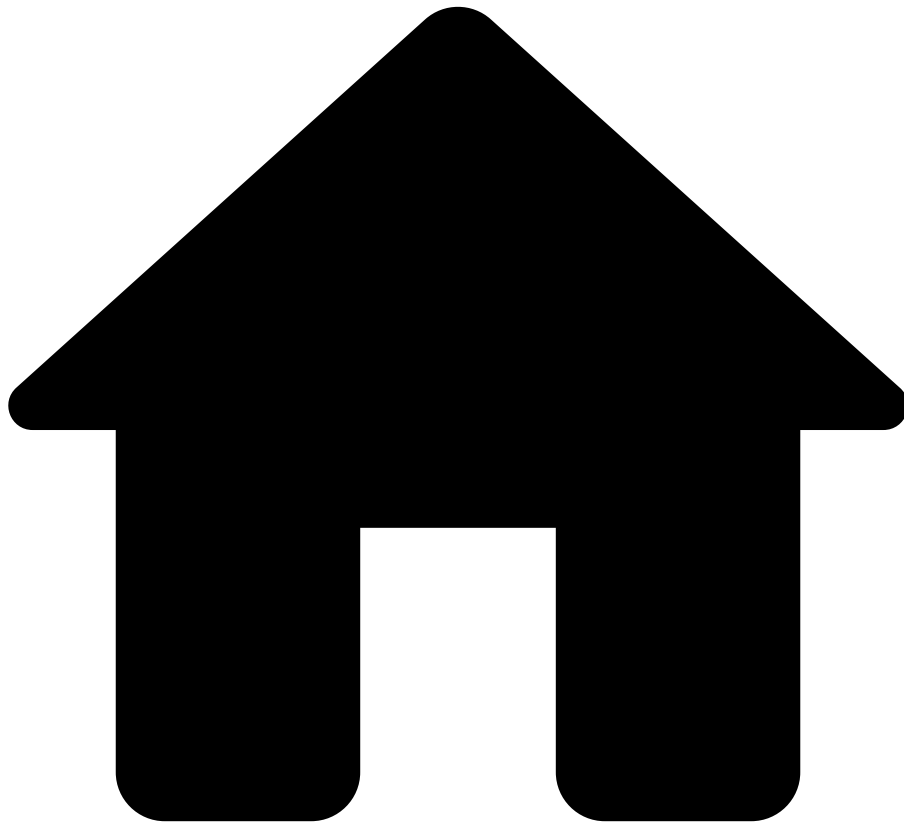
Types[â€‹](#)

- [Bool](#)
- [Collections](#)

Functions[â€‹](#)

- [all](#)
- [min](#)
- [max](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- bag

On this page

bag

```
bag(list: List<T>) : Bag<T>
```

Returns a bag containing the same elements as the input list, but without any ordering guarantees.

Examples[↗](#)

Basic Usage[↗](#)

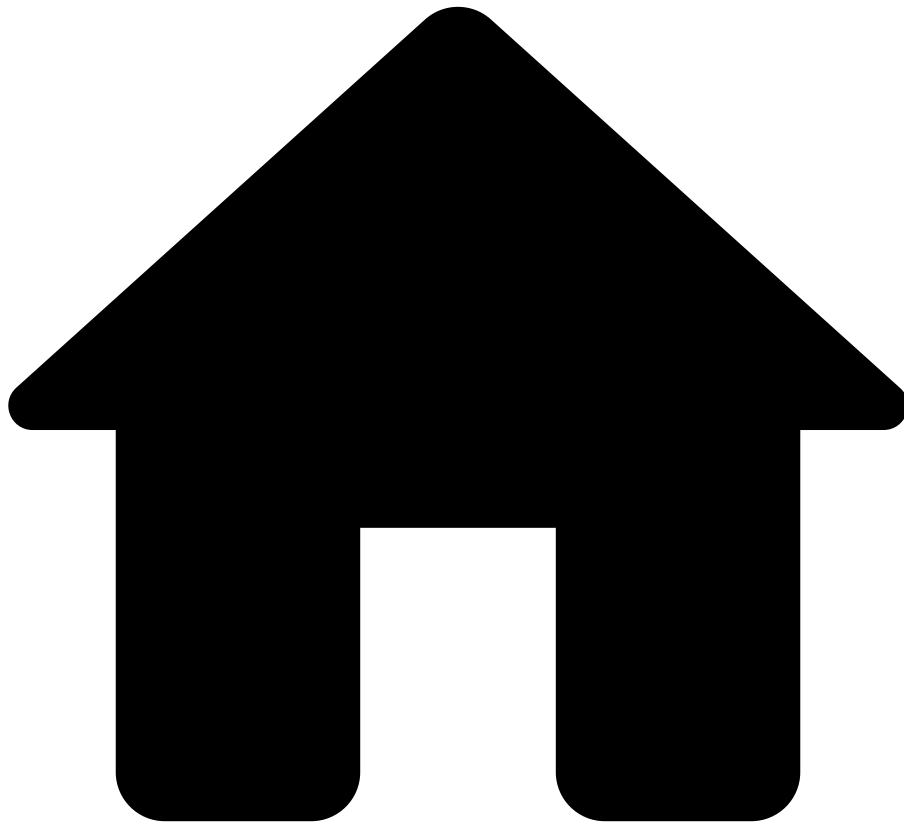
```
x = [3, 1, 2];  
y = bag(x); // Bag containing 1, 2, 3 (order undefined)
```

See Also[↗](#)

Guides[↗](#)

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering

.

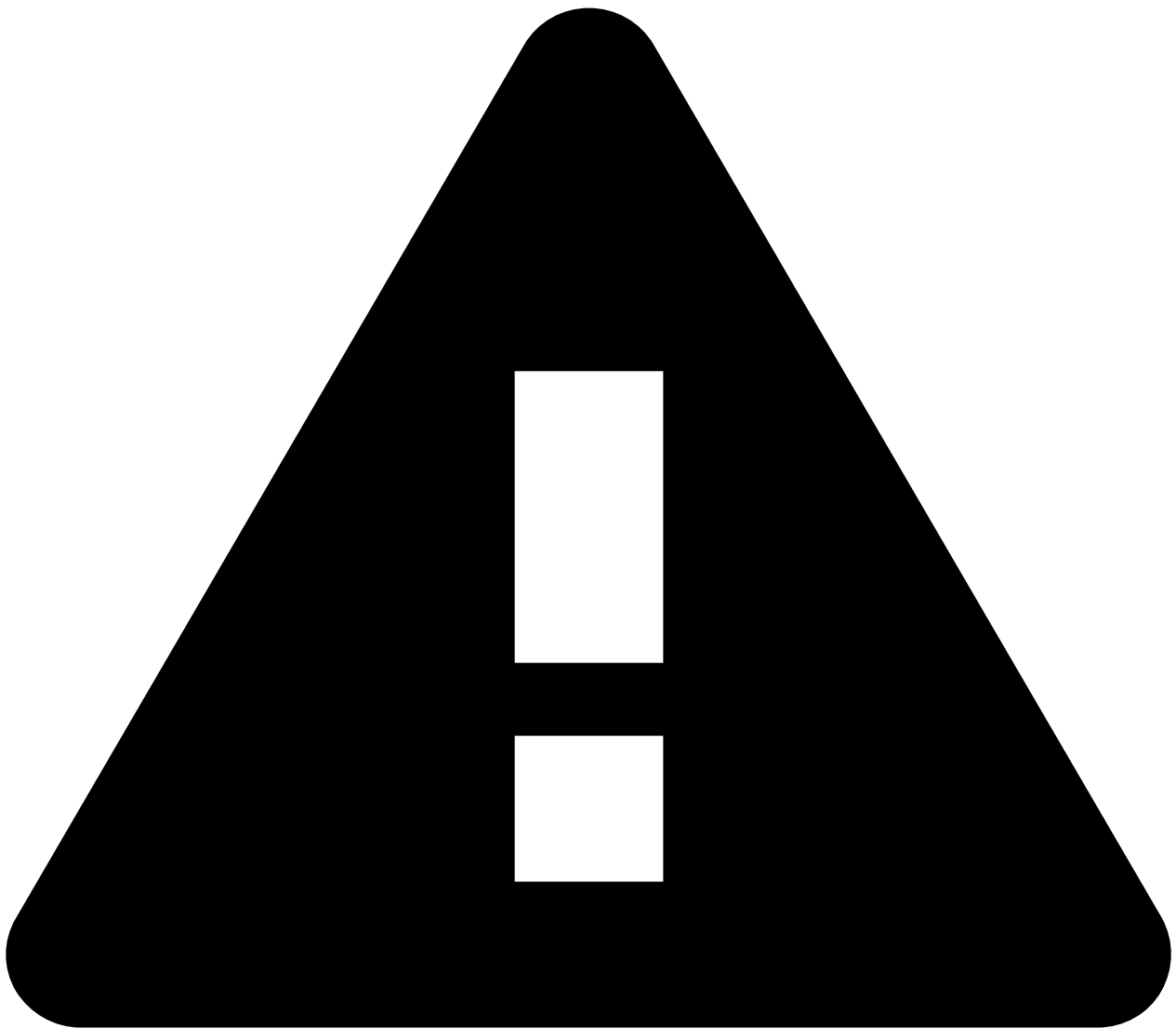


- Reference
- [NQE Language](#)
- [Standard Library](#)
- blockDiff

On this page

blockDiff

```
blockDiff(config: List<ConfigLine>, pattern: PatternBlocks<T>) :  
{data: T, blocks: MatchBlocks, diffCount: Integer}
```



Deprecated Bag overload

As a transitional measure for backwards compatibility, `blockDiff` also accepts a `Bag<ConfigLine>` as its first argument. Use `orderBy()` or `order()` to create an argument with type `List<ConfigLine>`. Eventually, the Bag overload will be removed and a type-checking error will be emitted instead.

See also

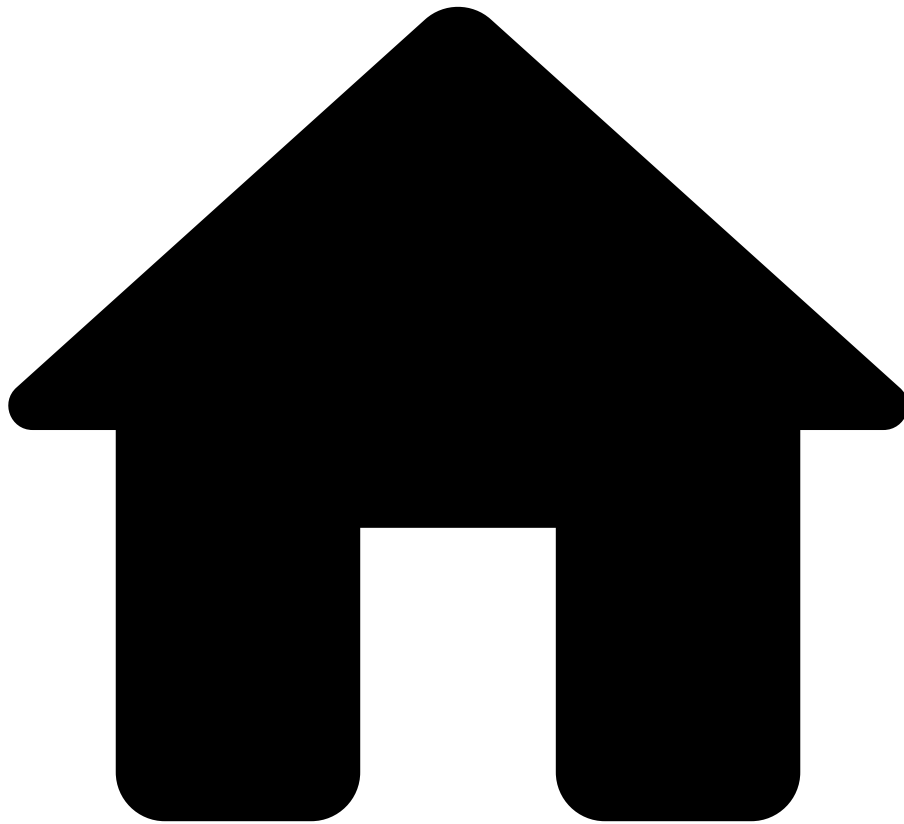
[ââ](#)

Functions

[ââ](#)

- [blockMatches](#)

.

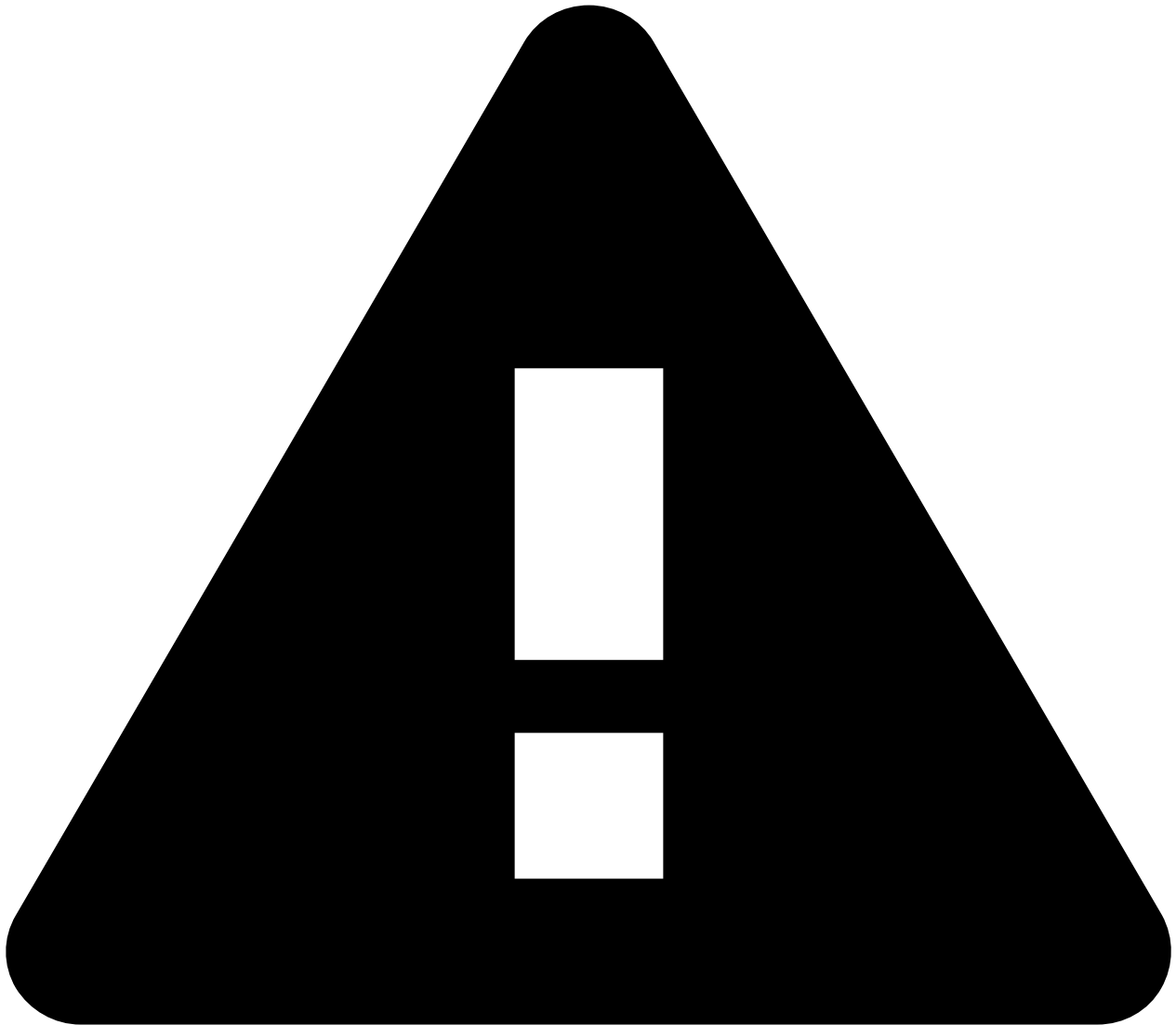


- Reference
- [NQE Language](#)
- [Standard Library](#)
- blockMatches

On this page

blockMatches

```
blockMatches(config: List<ConfigLine>, pattern:  
PatternBlocks<T>) : Bag<{data: T, blocks: MatchBlocks, diffCount:  
Integer}>
```



Deprecated Bag overload

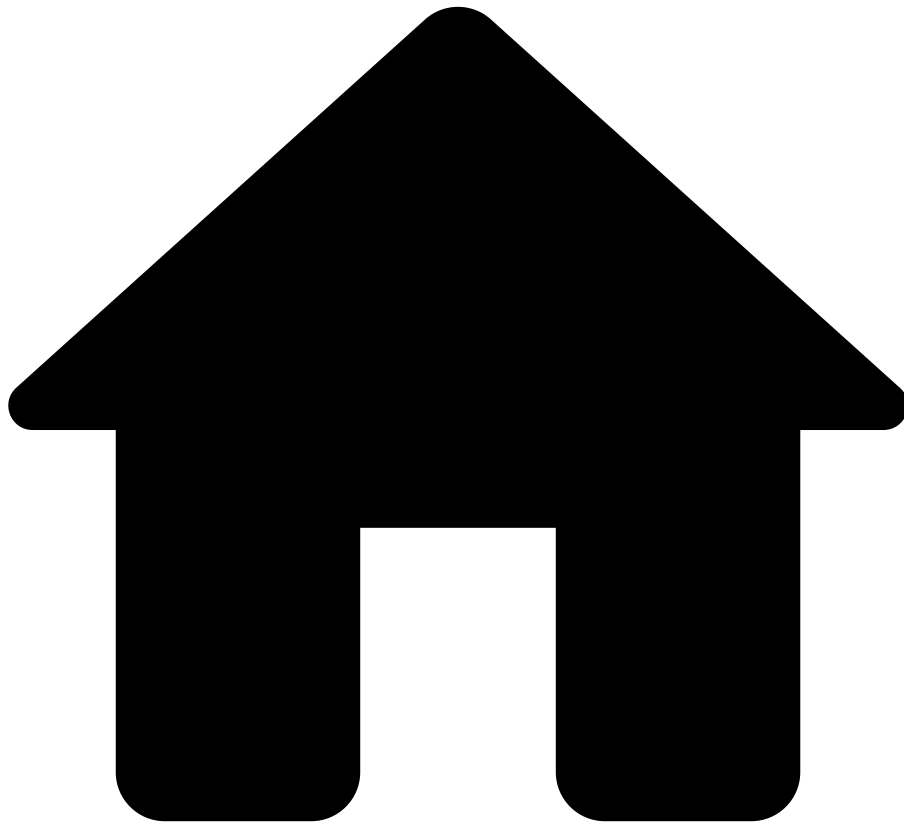
As a transitional measure for backwards compatibility, `blockMatches` also accepts a `Bag<ConfigLine>` as its first argument. Use `orderBy()` or `order()` to create an argument with type `List<ConfigLine>`. Eventually, the Bag overload will be removed and a type-checking error will be emitted instead.

See also [â](#)

Functions [â](#)

- [hasBlockMatch](#)
- [blockDiff](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- blockPattern

On this page

blockPattern

```
blockPattern(text: String) : PatternBlocks<{}>
```

The `blockPattern(text)` function can be used to parse a block pattern from a string. For example, the following query matches a set of patterns against device

configurations, where the patterns are obtained from HTTP sources using the `blockPattern` function:

```
foreach device in network.devices
  foreach httpSource in network.externalSources.httpSources
    foreach endpoint in httpSource.endpoints
      where endpoint.statusCode == 200
      let patternSource = endpoint.responseBody
      where isPresent(patternSource)
      foreach match
        in blockMatches(device.files.config,
          blockPattern(patternSource))
      select { deviceName: device.name, patternSource,
        matchedBlocks: match.blocks }
```

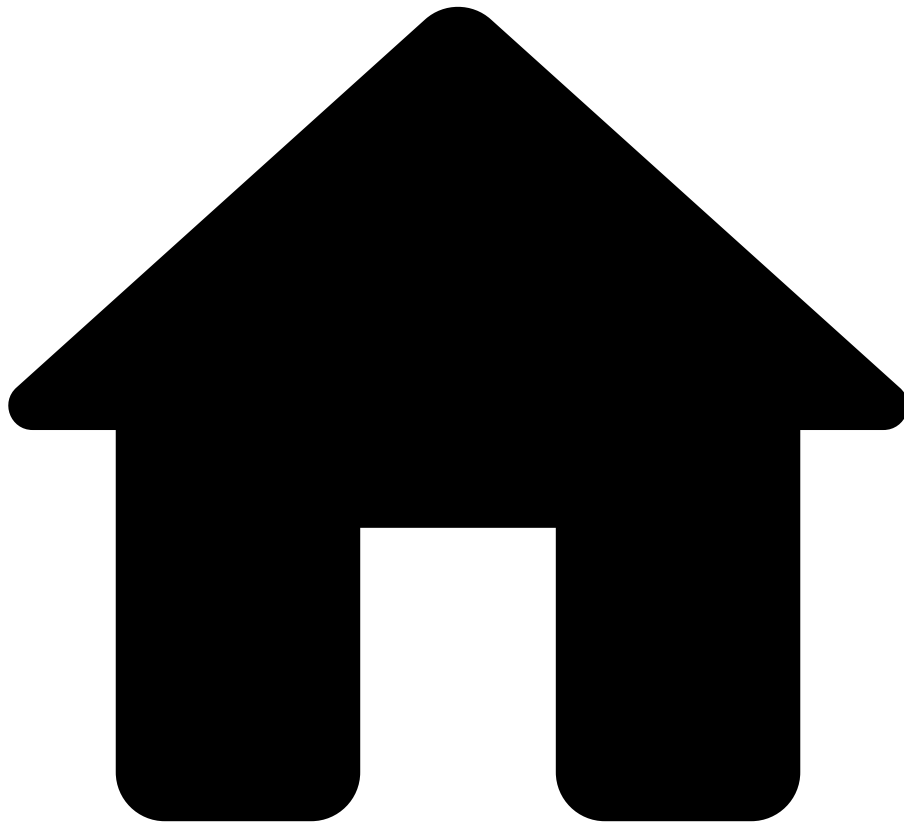
Note that the pattern source must not contain any capture groups.

See also [â](#)

Types [â](#)

- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- broadcastAddress

On this page

broadcastAddress

```
broadcastAddress(subnet: IpSubnet) : IpAddress
```

Returns the broadcast IP address for a subnet. That is, the IP address for a subnet, with all host bits set to 1. For example,

```
broadcastAddress(ipSubnet("1.2.3.4/24")) is  
ipAddress("1.2.3.255") .
```

See also[â](#)

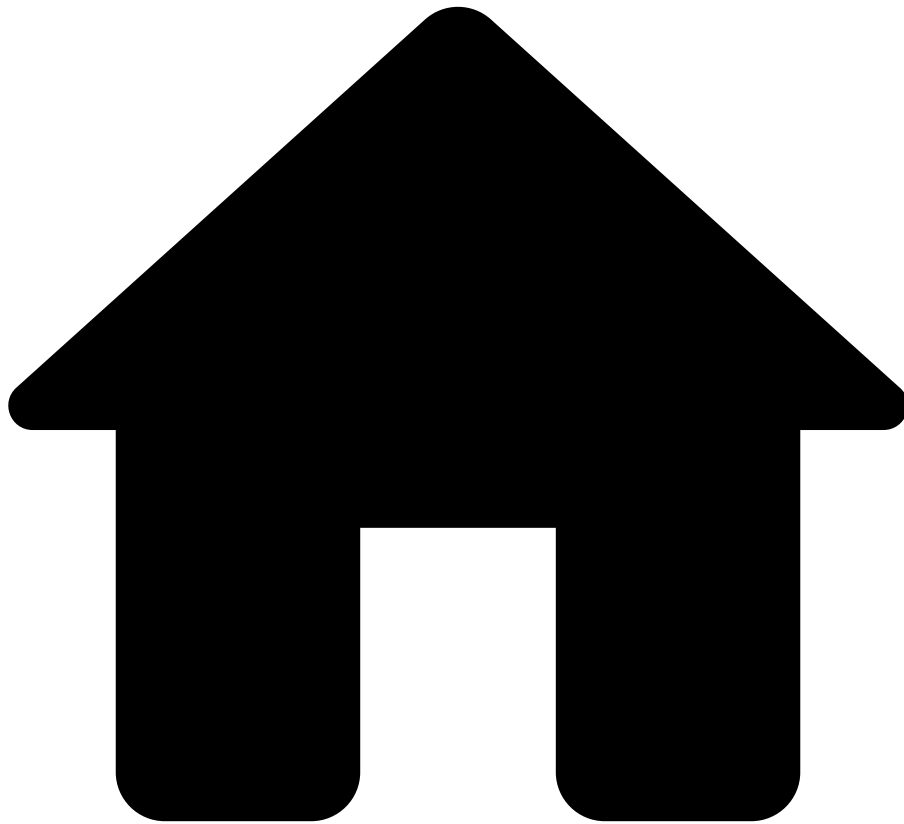
Types[â](#)

- [IpAddress](#)
- [IpSubnet](#)

Functions[â](#)

- [ipAddress](#)
- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ceil

On this page

ceil

```
ceil(number: Float) : Integer
```

Gets an integer from a float by *rounding up* to the nearest larger integer; e.g.,

```
int(2.4) results in 3 .
```

See also[ââ](#)

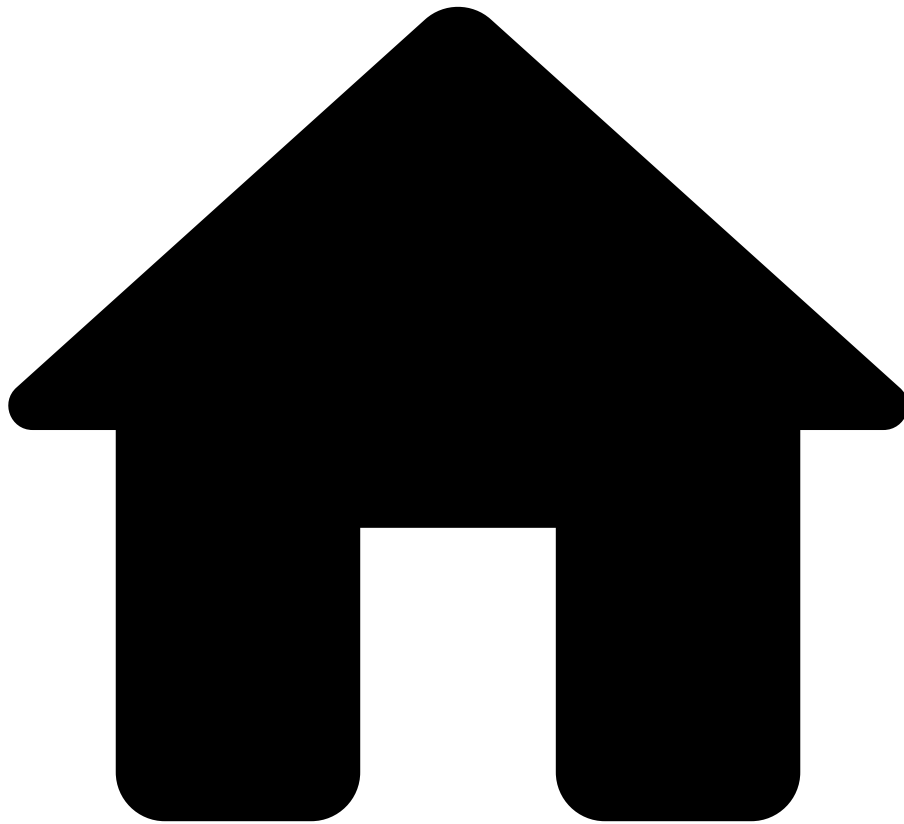
Functions[ââ](#)

- [float](#)
- [int](#)
- [floor](#)
- [round](#)
- [toNumber](#)

Types[ââ](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- date

On this page

date

`date(text: String) : Date`

Obtains a `Date` from a `String` that represents a date according to the ISO 8601 standard. For example, `date("1929-10-29")` returns a `Date` that represents October 29, 1929.

This function will raise an error if the `String` deviates from the ISO 8601 standard for dates or if the `String` is absent.

```
date(timestamp: Timestamp) : Date
```

Obtains a `Date` from a `Timestamp` by returning the date portion of a `Timestamp` when placed in the UTC timezone.

This function will raise an error if the resulting date exceeds the range of `Date` or if the `Timestamp` is absent.

See also

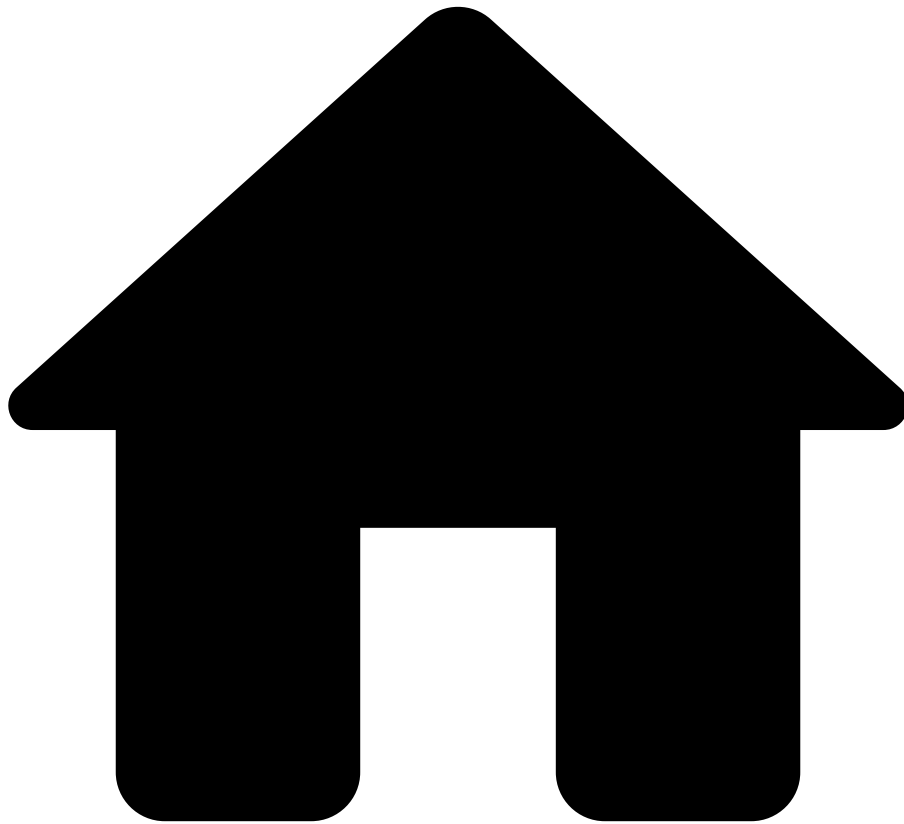
Functions

- [timestamp](#)
- [duration](#)
- [days](#)
- [hours](#)
- [minutes](#)
- [seconds](#)

Types

- [Time](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- days

On this page

days

`days(value: Integer) : Duration`

Obtains a `Duration` representing the given number of days. For example, `days(3)` returns a `Duration` that represents three days. The value can be negative. For example, `days(-2)` returns a `Duration` that represents negative two days.

This function will raise an error if the absolute value of the input is greater than `106751991167300` .

See also[â](#)

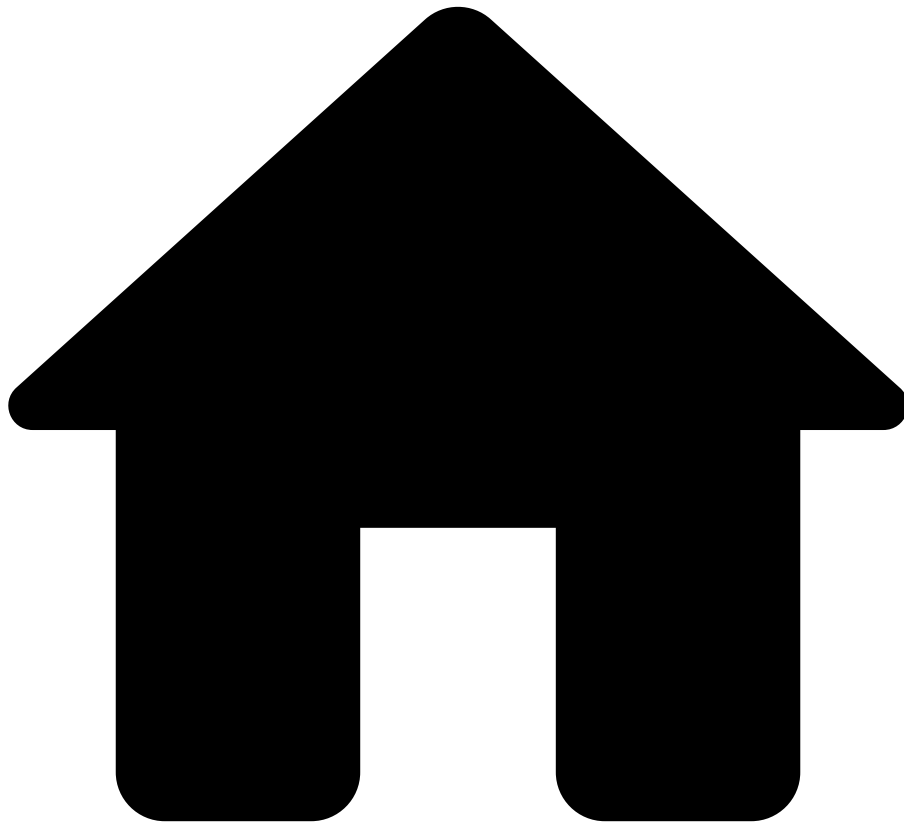
Functions[â](#)

- [date](#)
- [timestamp](#)
- [duration](#)
- [hours](#)
- [minutes](#)
- [seconds](#)

Types[â](#)

- [Time](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- distinct

On this page

distinct

For any type `T`,

```
distinct(collection: List<T>) : List<T>  
distinct(collection: Bag<T>)  : Bag<T>
```

Returns the unique items in `collection` .

- For lists, `distinct` deduplicates the collection while preserving the order of the items in the original list: The first occurrence of an element is kept and all other subsequent occurrences are dropped.

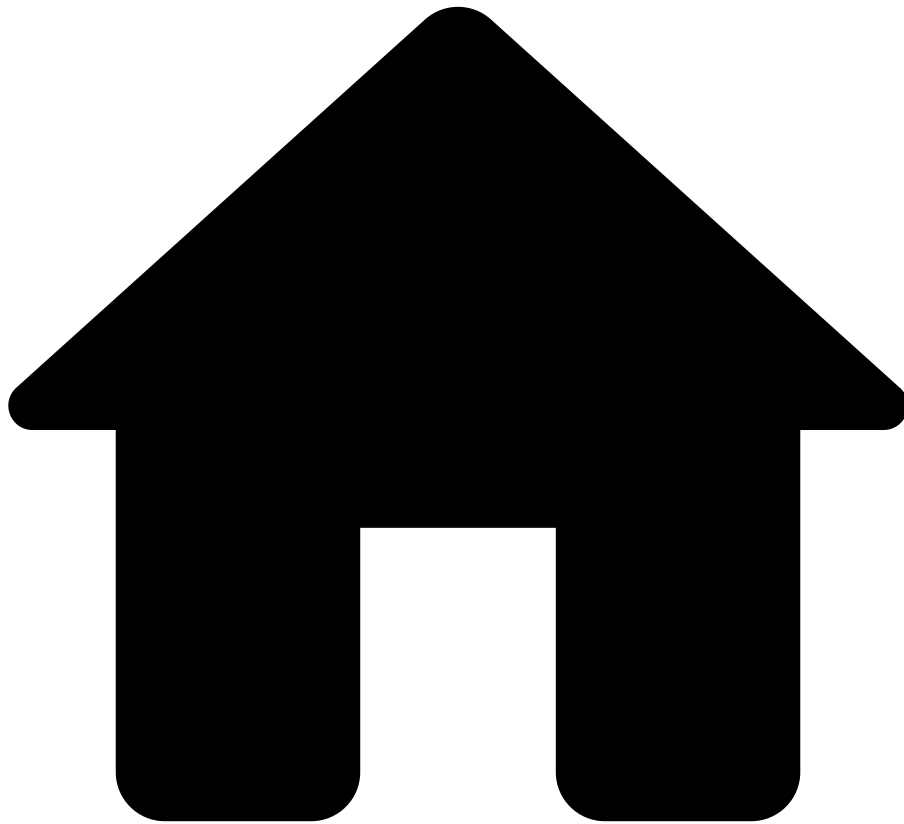
```
list      = [1, 2, 3, 1, 4, 2];  
uniqueList = distinct(list); // [1, 2, 3, 4]
```

See also[â](#)

Types[â](#)

- [Collections](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- duration

On this page

duration

`duration(text: String) : Duration`

Obtains a `Duration` from a `String` that represents a duration according to the ISO 8601 standard. For example, `duration("PT23H56M4S")` returns a `Duration` that represents 23 hours, 56 minutes, and 4 seconds.

This function will raise an error if the string significantly deviates from the ISO 8601 standard. A permitted deviation is for strings that represent a negative interval of time. This function accepts strings like `"-P1DT12H"`, `"P-1DT-12H"`, and `"P-2DT12H"`, which all represent -36 hours.

This function will also raise an error if the string represents a duration with a nonzero amount of nanoseconds, such as the string `"PT0.5S"`.

See also

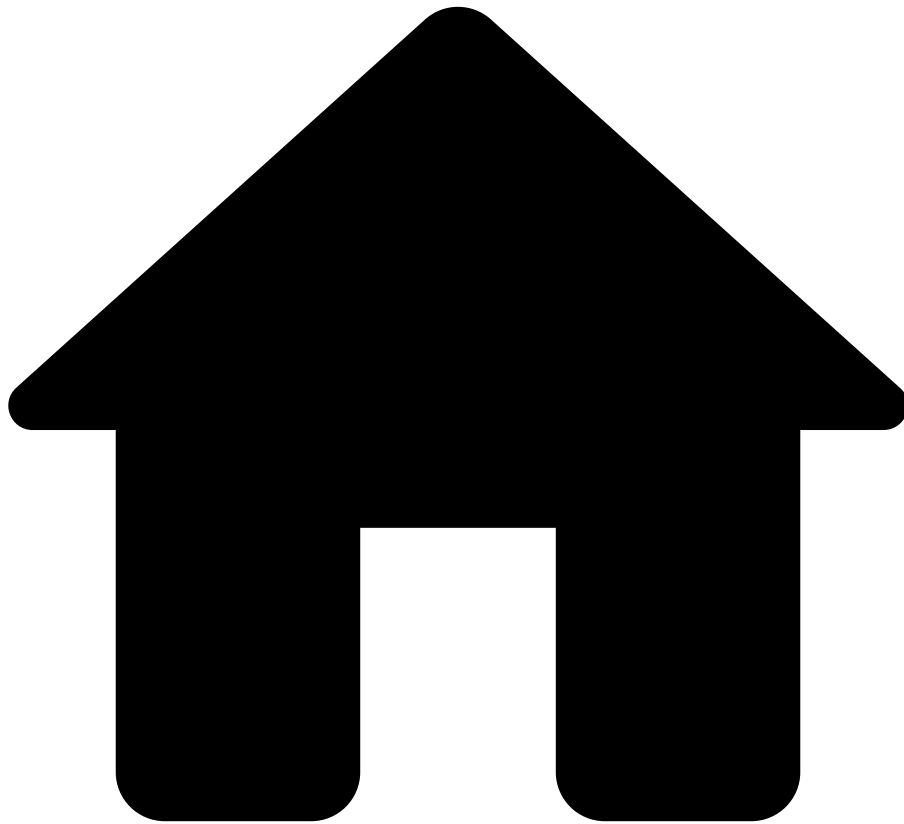
Functions

- [date](#)
- [timestamp](#)
- [days](#)
- [hours](#)
- [minutes](#)
- [seconds](#)

Types

- [Time](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `extractJson`

extractJson

`extractJson[T](json: String) : T` for most types `T`. One example that `T` cannot be nor can contain is some `OneOf` type.

NQE values can be extracted from a JSON string using the `extractJson` function.

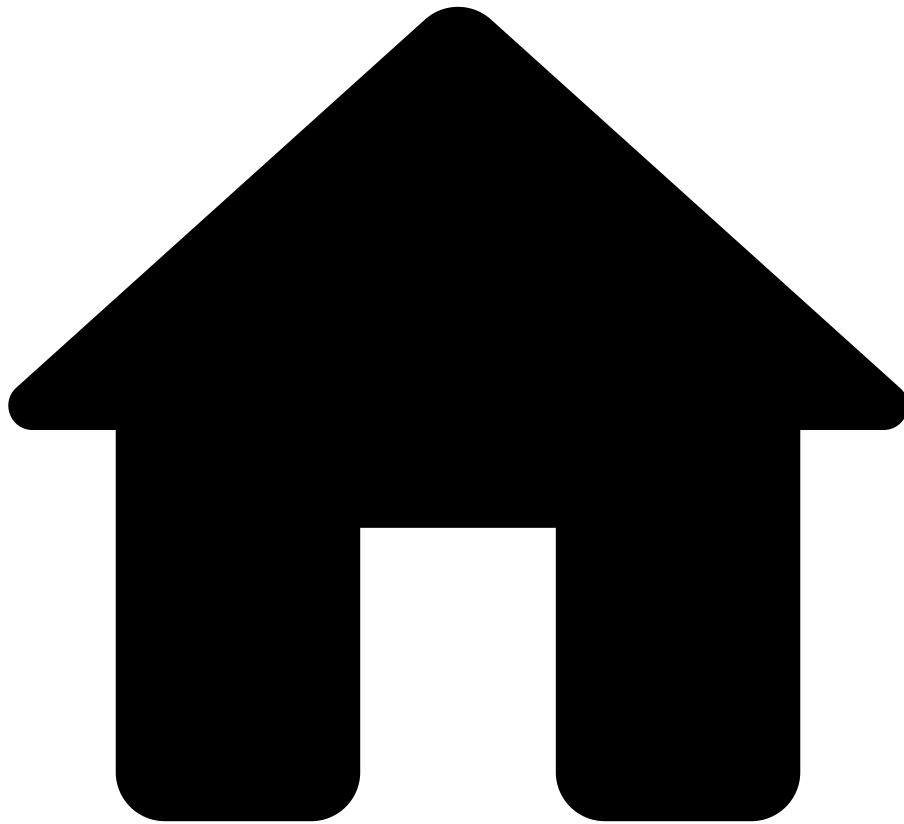
```
extractJson[Integer]("1")
```

The above expression will return the NQE value `1` of type `Integer`. The `String` value that contains the JSON value (`"1"` above) is passed to `extractJson` as a regular argument. The type of NQE value to be extracted from the JSON value is written between square brackets prior to the function call's parentheses (`Integer` in the above example).

The following are examples of expressions with `extractJson` calls and the NQE value that results from evaluating each expression.

- `extractJson[Integer]("1") => 1`
- `extractJson[Boolean]("true") => true`
- `extractJson[String]("\"foo\") => \"foo\"`
- `extractJson[IpAddress]("\"192.168.0.1\") => 192.168.0.1`
- `extractJson[{}](\"{\}") => {}`
- `extractJson[{}](\"{\\\"x\\\" : 1}\") => {}`
- `extractJson[{x: Integer}](\"{\\\"x\\\" : 1}\") => { x: 1 }`
- `extractJson[{x: Integer, y: Boolean}](\"{\\\"x\\\": 1, \\\"y\\\": true}\") => { x: 1, y: true }`
- `extractJson[{x?: Integer}](\"{\}") => { x: null }`
- `extractJson[{x?: Integer}](\"{\\\"x\\\": null}\") => { x: null }`
- `extractJson[List<Integer>](\"[1,2]\") => [1, 2]`
- `extractJson[Bag<Integer>](\"[1,2]\") => bag([1, 2])`
- `extractJson[List<{x: Integer}>](\"[{\\\"x\\\":1},{\\\"x\\\":2}]\") => [{ x: 1 }, { x: 2 }]`

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- findInterface

findInterface

```
findInterface(device: Device, ifaceName: String) : { interface: Iface, subInterface: SubInterface?}?
```

A single device's configuration and state files may refer to the same interface by slightly different names. For example, on a Cisco NXOS device, a port channel might appear in the configuration as "interface port- channel3", while the output of a "show interface"

command on the device may refer to the same interface as "Po3", using an abbreviated form and different capitalization of letters.

The normalized data model for interfaces uses a canonical, normalized name for each interface. The `findInterface` function enables you to lookup an interface in the normalized data model using a name that might not be normalized; for example, you can use it to look up an interface using "port-channel3" as it appears in the config, even though it is named "po3" in the normalized data model.

Specifically, `findInterface` takes two arguments: a device and an interface name. For example, `findInterface(device, "port-channel3")` returns:

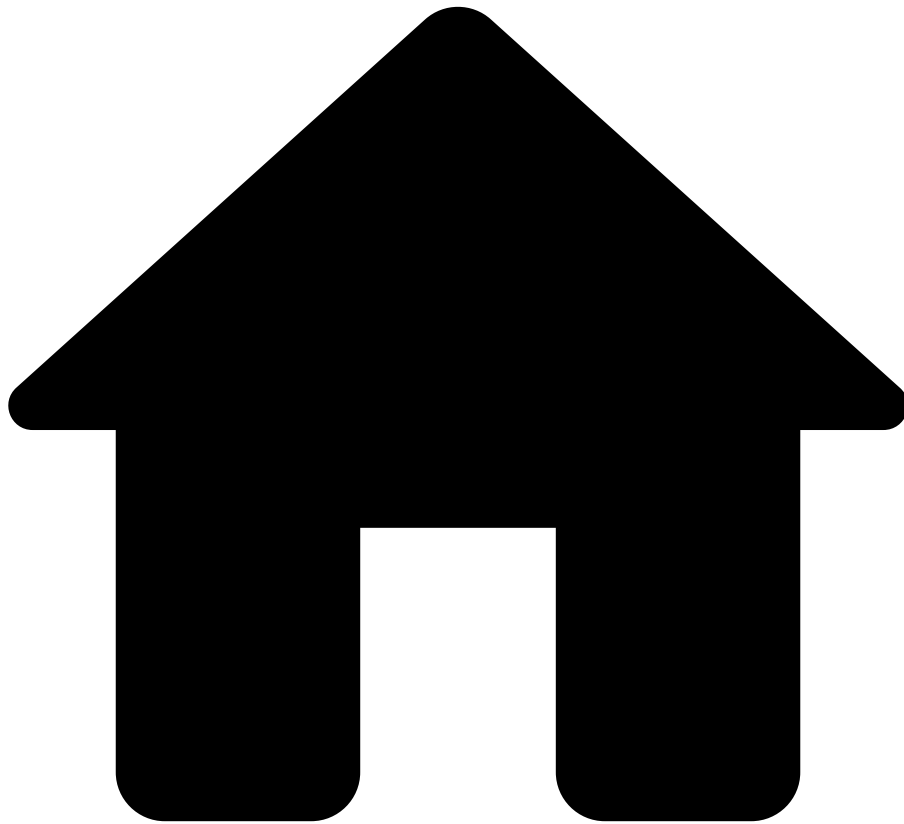
- `null` if "port-channel3" returned no matches across the device's interfaces and the sub-interfaces of all the device's interfaces.
- `{ interface: Iface, subInterface: null }` if "port-channel3" matched one of the device's interfaces, but none of that interface's sub-interfaces were matched.
- `{ interface: Iface, subInterface: SubInterface }` if "port-channel3" matched one of the device's interfaces' sub-interfaces. In this case the matched sub-interface's parent interface is also returned.

As an example, the following query find interfaces on NXOS devices that have some ip ospf configuration and MTU setting above 9000. The query finds relevant interfaces in the configuration, and then uses `findInterface` to lookup the MTU setting in the normalized data model based on the interface name as it appears in the configuration.

```
foreach device in network.devices
where device.platform.os == OS.NXOS
foreach match in patternMatches(device.files.config,
`interface {ifaceName:string}`)
where length(patternMatches(match.line.children, `ip ospf`))
> 0
let ifaceMatch = findInterface(device, match.data.ifaceName)
where isPresent(ifaceMatch) && ifaceMatch.interface.mtu >
9000
select {
  deviceName: device.name,
  iface: ifaceMatch.interface.name,
```

```
mtu: ifaceMatch.interface.mtu  
}
```

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- float

On this page

float

```
float(number: Integer) : Float
```

Converts an integer-valued expression to a floating-point number.

Some examples, where `x` and `y` are assumed to be variables that have integers assigned to them:

- `float(x)`
- `float((x + y) / 100.)`
- `float(sum(fromTo(1,100)))`

See also [â](#)

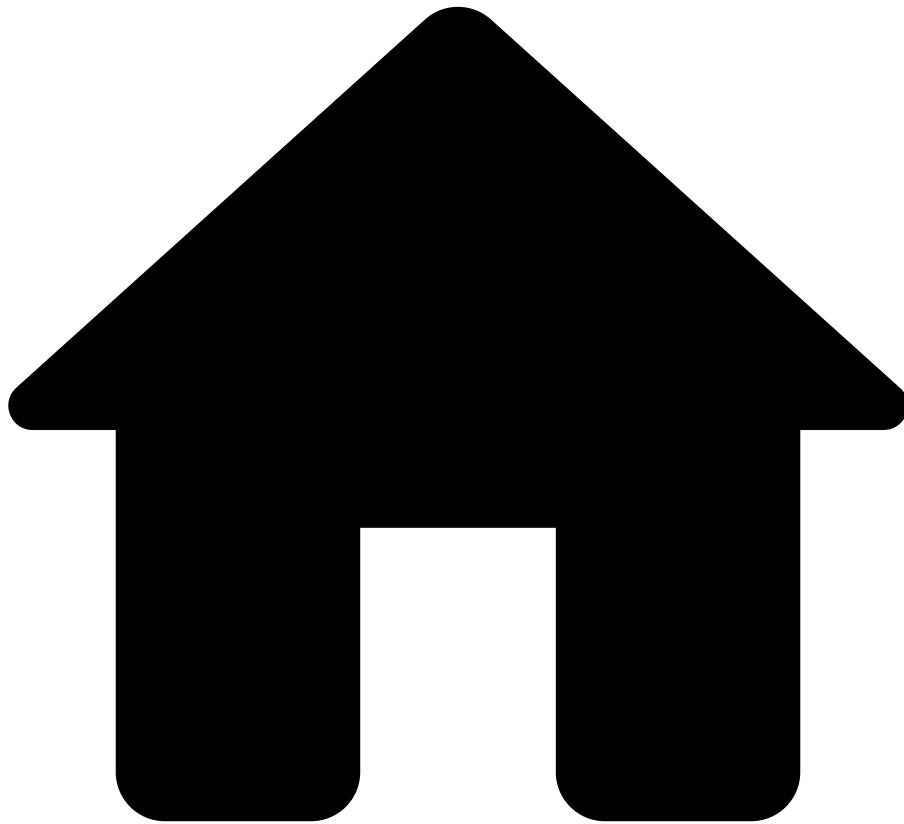
Functions [â](#)

- [int](#)
- [floor](#)
- [ceil](#)
- [round](#)

Types [â](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- floor

On this page

floor

`floor(number: Float) : Integer`

Gets an integer from a float by *rounding down* to the nearest lower; e.g., `int(2.9)` results in `2`.

See also[ââ](#)

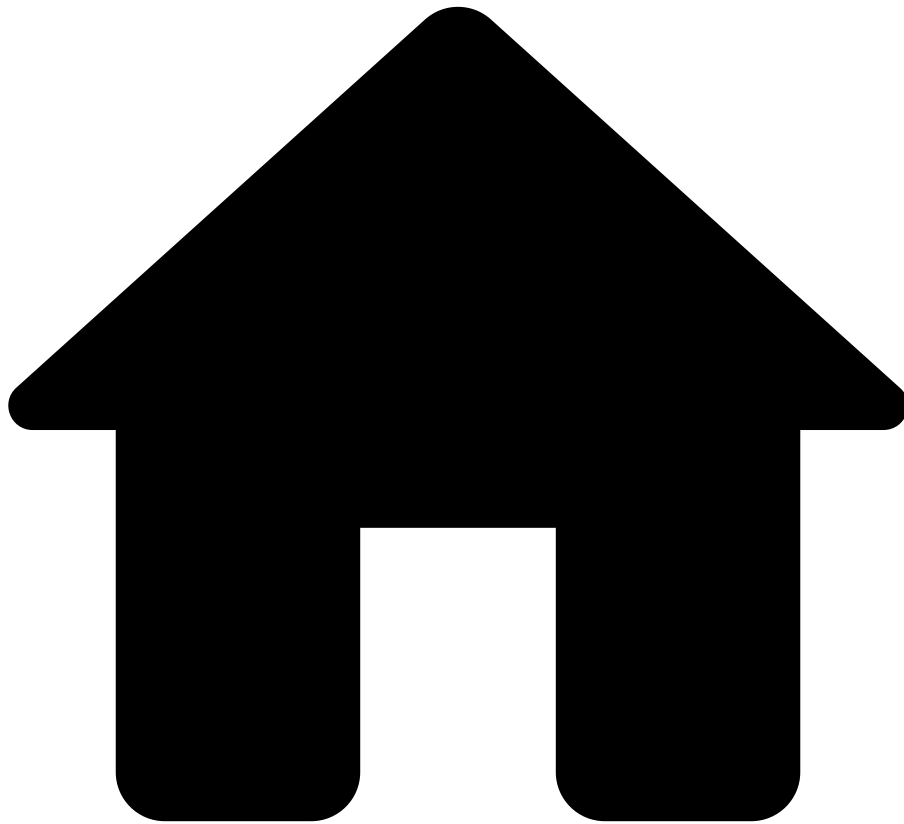
Functions[ââ](#)

- [float](#)
- [int](#)
- [ceil](#)
- [round](#)
- [toNumber](#)

Types[ââ](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- fromTo

On this page

fromTo

```
fromTo(a: Integer, b: Integer) : List<Integer>
```

Returns an ordered list of consecutive integers from `a` to `b` (inclusive). The list is in ascending order.

- For example, `fromTo(1, 3)` returns `[1, 2, 3]`.
- If the second argument is smaller than the first, as in `fromTo(3, 1)`, the returned value is the empty list.

```
fromTo(a: IPAddress, b: IPAddress) : List<IPAddress>
```

A list of IP addresses in a range can be constructed with `fromTo(a, b)`, which equals the list of IP addresses starting from `a` and ending with `b`. For example:

- `fromTo(ipAddress("1.2.2.254"), ipAddress("1.2.3.1"))` is same as `[1.2.2.254, 1.2.2.255, 1.2.3.0, 1.2.3.1]`
- `fromTo(ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:ffff"), ipAddress("2001:0db8:85a3:0000:0000:8a2e:0371:001"))` is same as `[2001:0db8:85a3:0000:0000:8a2e:0370:ffff, 2001:0db8:85a3:0000:0000:8a2e:0370:ffff, 2001:0db8:85a3:0000:0000:8a2e:0371:0, 2001:0db8:85a3:0000:0000:8a2e:0371:1]`.

If the second argument is smaller than the first, as in

`fromTo(ipAddress("1.2.2.4"), ipAddress("0.0.0.0"))`, the returned value is the empty list.

This function will raise an error:

- if `a` and `b` are not of same IP address family. For example, `fromTo(ipAddress("1.2.2.254"), ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:ffff"))`.
- if number of IP addresses between `a` and `b` is 2147483647 or greater.

See also

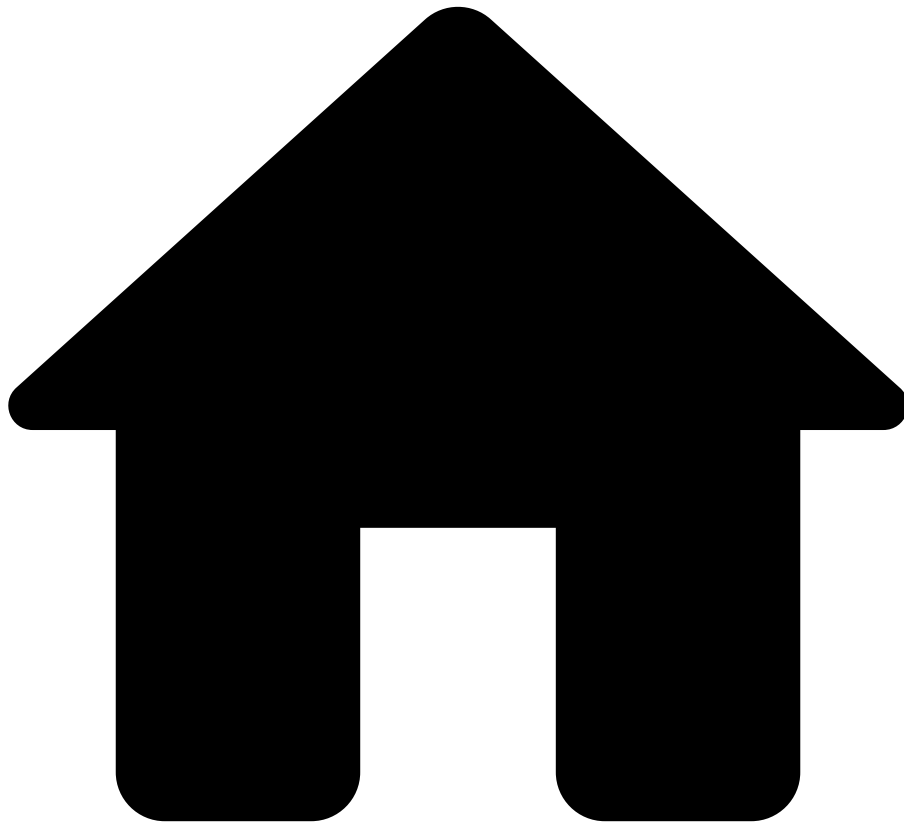
Guides

- [Collections](#) - Working with ordered lists
- [Integer](#)
- [IpAddress](#)

Functions

- [ipAddress](#) - Create IP address from string

.



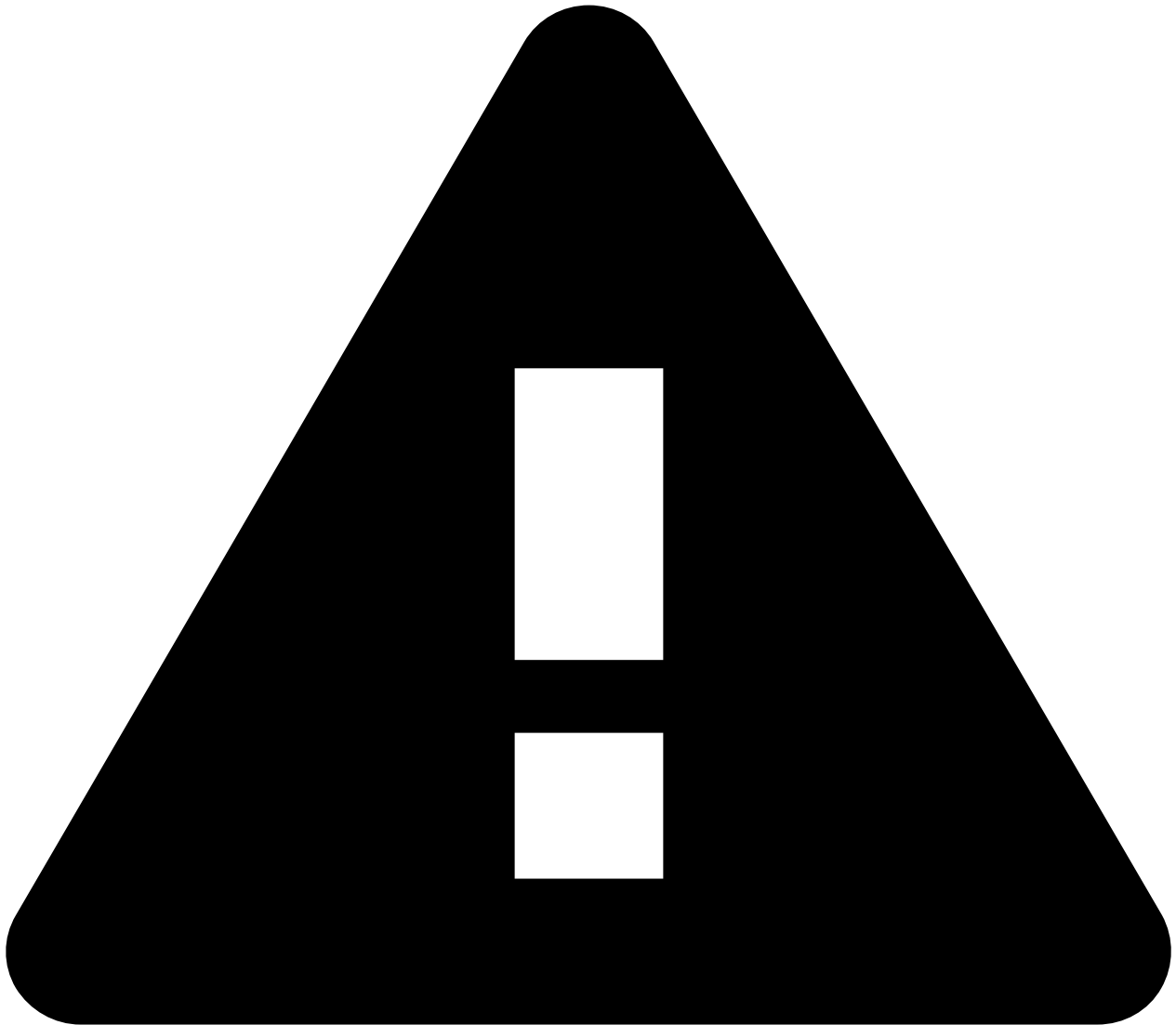
- Reference
- [NQE Language](#)
- [Standard Library](#)
- hasBlockMatch

On this page

hasBlockMatch

```
hasBlockMatch(config: List<ConfigLine>, pattern:  
PatternBlocks<T>) : Bool
```

If [blockMatches](#) returns a nonempty collection on the same input, then this function returns `true`. Otherwise, this function returns `false`.



Deprecated Bag overload

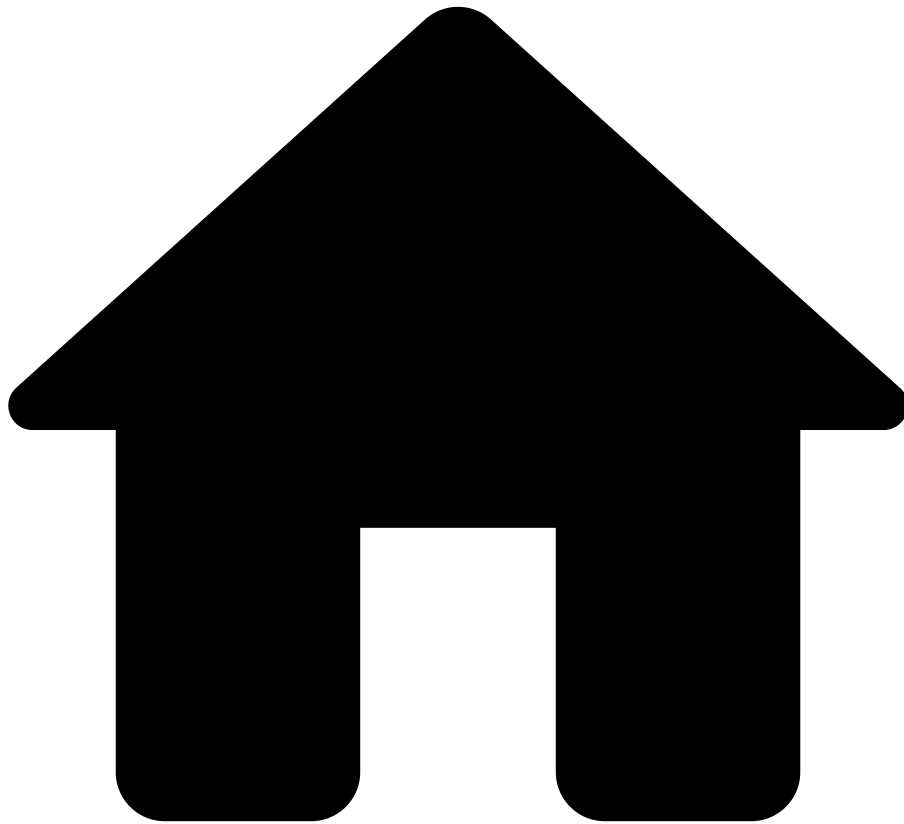
As a transitionary measure for backwards compatibility, `hasBlockMatch` also accepts a `Bag<ConfigLine>` as its first argument. Use `orderBy()` or `order()` to create an argument with type `List<ConfigLine>`. Eventually, the Bag overload will be removed and a type-checking error will be emitted instead.

See also [â](#)

Functions [â](#)

- [blockMatches](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- hasMatch

On this page

hasMatch

```
hasMatch(text: String, regex: Regex<T>) : Bool
```

Checks whether `text` matches the structure specified by `regex`. The entirety of `text` must match.

Examples

Example	Result
<code>hasMatch("", re``)</code>	true
<code>hasMatch(" ", re``)</code>	false
<code>hasMatch("Name: Jane, Age: 32", re`Name: (\w+), Age: (\d+)`)</code>	true
<code>hasMatch("", re`Name: (\w+), Age: (\d+)`)</code>	false
<code>hasMatch("192.168.0.1", re`\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b`)</code>	true
<code>hasMatch("1.2.3x4", re`\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b`)</code>	false
<code>hasMatch("device-config.txt", re`[^ ]*\.txt`)</code>	true
<code>hasMatch("device config.org", re`[^ ]*\.txt`)</code>	false
<code>hasMatch("2357", re`(?<age:Integer>\w+)`)</code>	true
<code>hasMatch("3x", re`(?<age:Integer>\w+)`)</code>	false
<code>hasMatch("3.14", re`(?<age:Float>\d+\.\d+)`)</code>	true
<code>hasMatch("3.14", re`(?<age:Float>\w+)`)</code>	false
<code>hasMatch("2.4.6.8", re`(?<age:IpAddress>\d.\d.\d.\d)`)</code>	true
<code>hasMatch("a.b.c.d", re`(?<age:IpAddress>\w+)`)</code>	false

See also

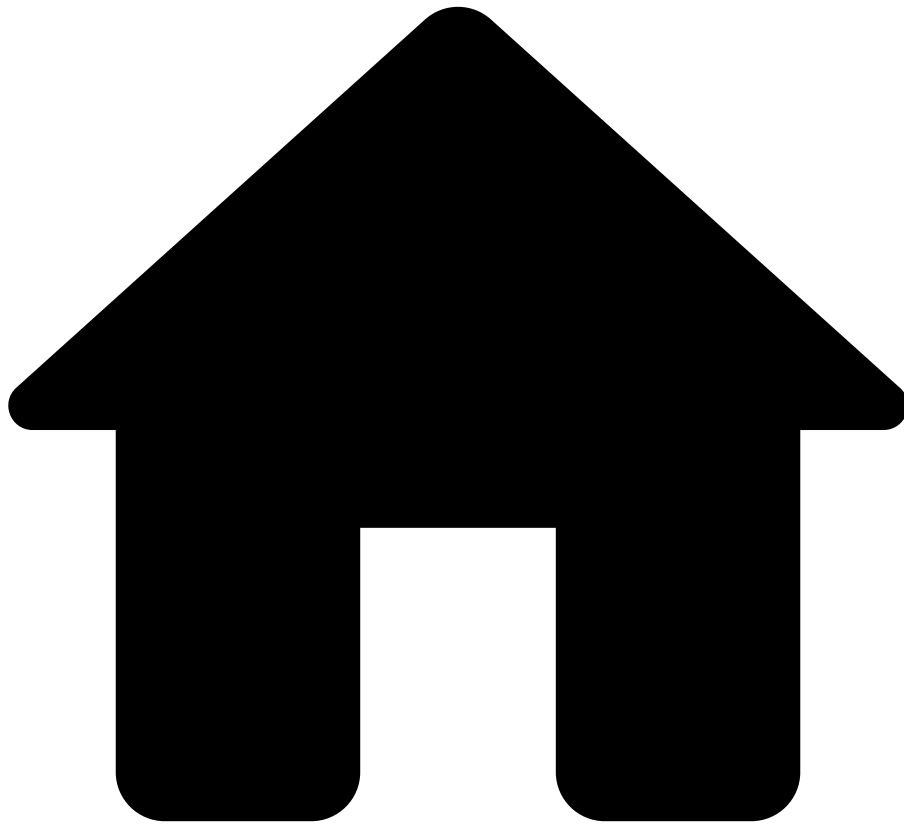
Functions

- [match](#): Gets a record of information about text that matches the structure specified by a regex, or otherwise `null`.
- [regexMatches](#): Gets the list of all substring matches of a given regex.

Types [☞☞](#)

See [the regular expression guide](#).

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- hours

On this page

hours

`hours(value: Integer) : Duration`

Obtains a `Duration` representing the given number of hours. For example, `hours(3)` returns a `Duration` that represents three hours. The value can be

negative. For example, `hours (-2)` returns a `Duration` that represents negative two hours.

This function will raise an error if the absolute value of the input is greater than `2562047788015215`.

See also

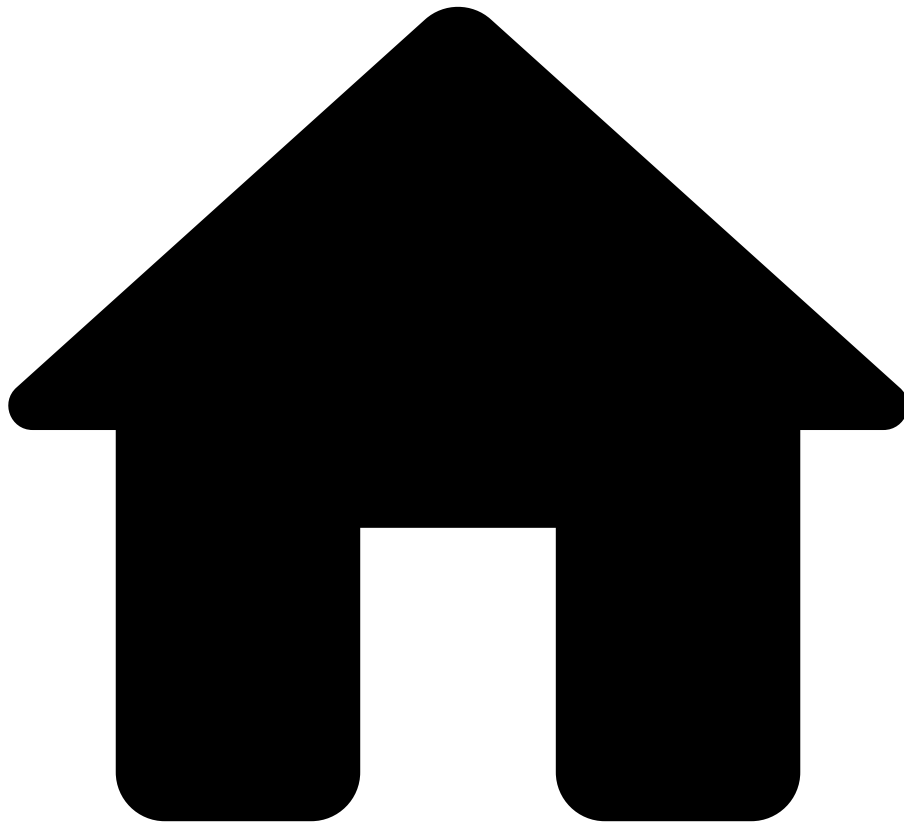
Functions

- [date](#)
- [timestamp](#)
- [duration](#)
- [days](#)
- [minutes](#)
- [seconds](#)

Types

- [Time](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- int

On this page

int

```
int(number: Float) : Integer
```

Truncates a floating-point number to obtain an integer; e.g., `int(2.9)` results in `2`.

If we have a named *float* `x`, then we can convert it to an *integer* by writing `int(x)`.

See also [ââ](#)

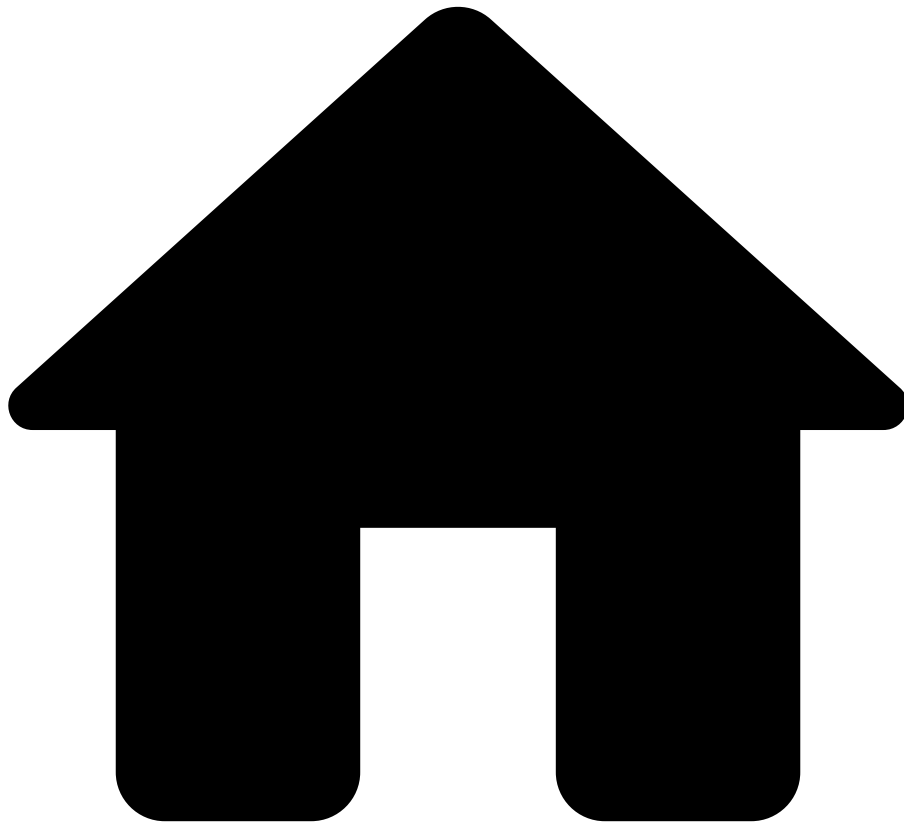
Functions [ââ](#)

- [float](#)
- [floor](#)
- [ceil](#)
- [round](#)
- [toNumber](#)

Types [ââ](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- intersect

On this page

intersect

`intersect(set1: Set<T>, set2: Set<T>) : Set<T>` for any type `T`.

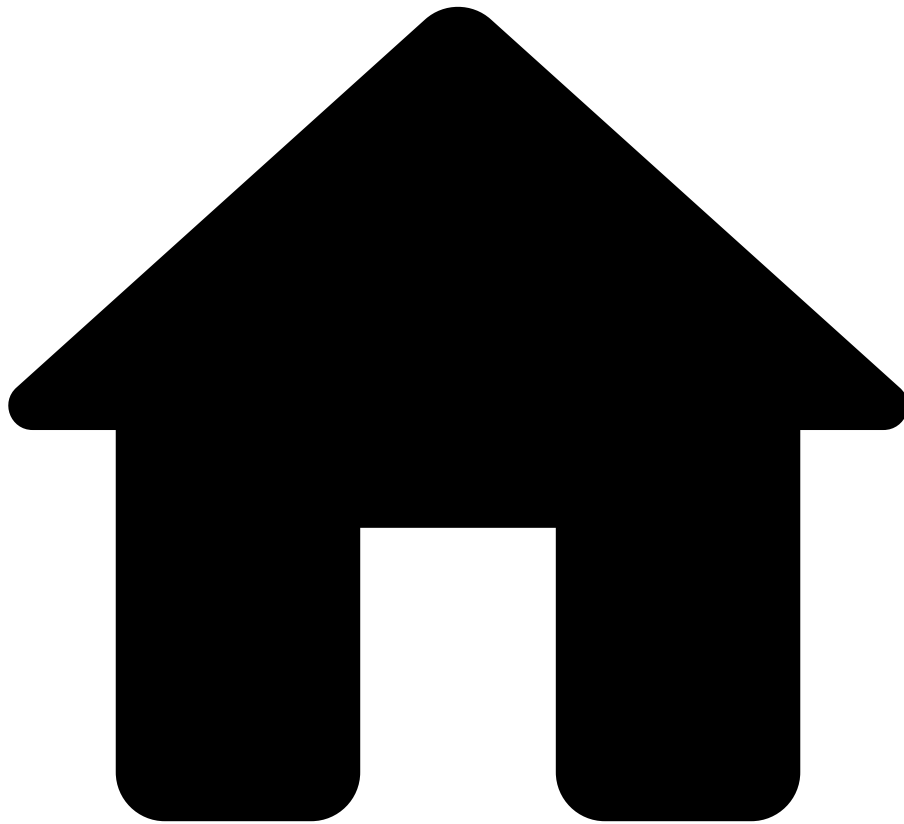
Returns the intersection of `set1` and `set2`.

See also [â](#)

Types [â](#)

- [Set](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipAddress

On this page

ipAddress

```
ipAddress(str: String) : IPAddress
```

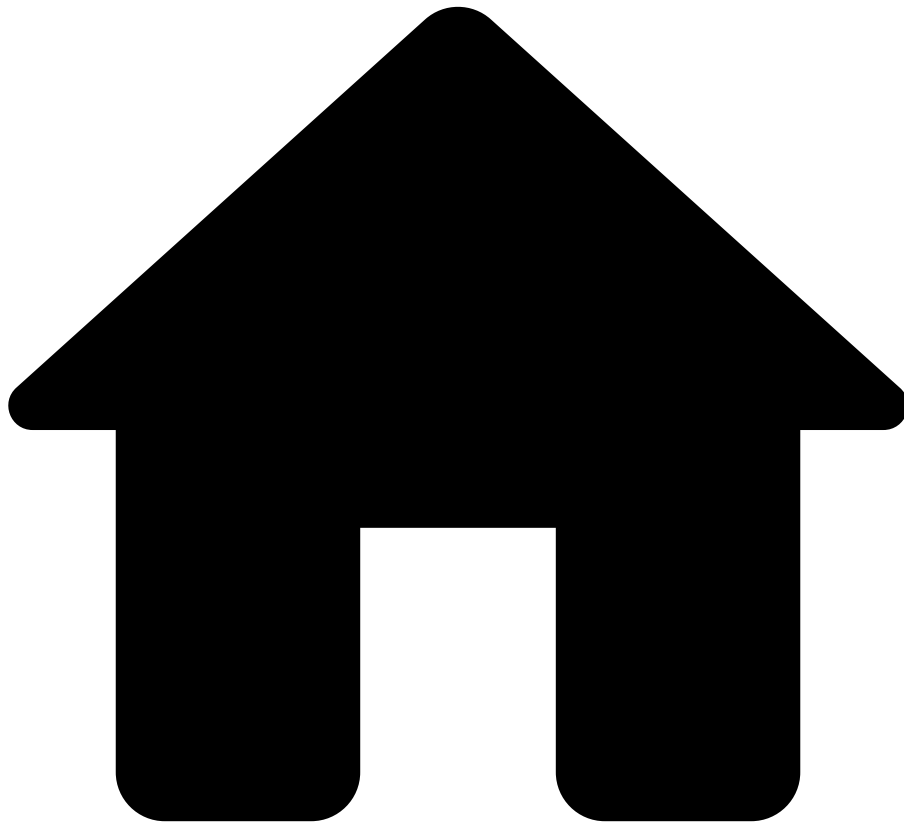
An IP address can be written like this: `ipAddress("1.2.3.4")`.

See also [a](#)

Types [a](#)

- [ipv4Address](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipAddressSet

On this page

ipAddressSet

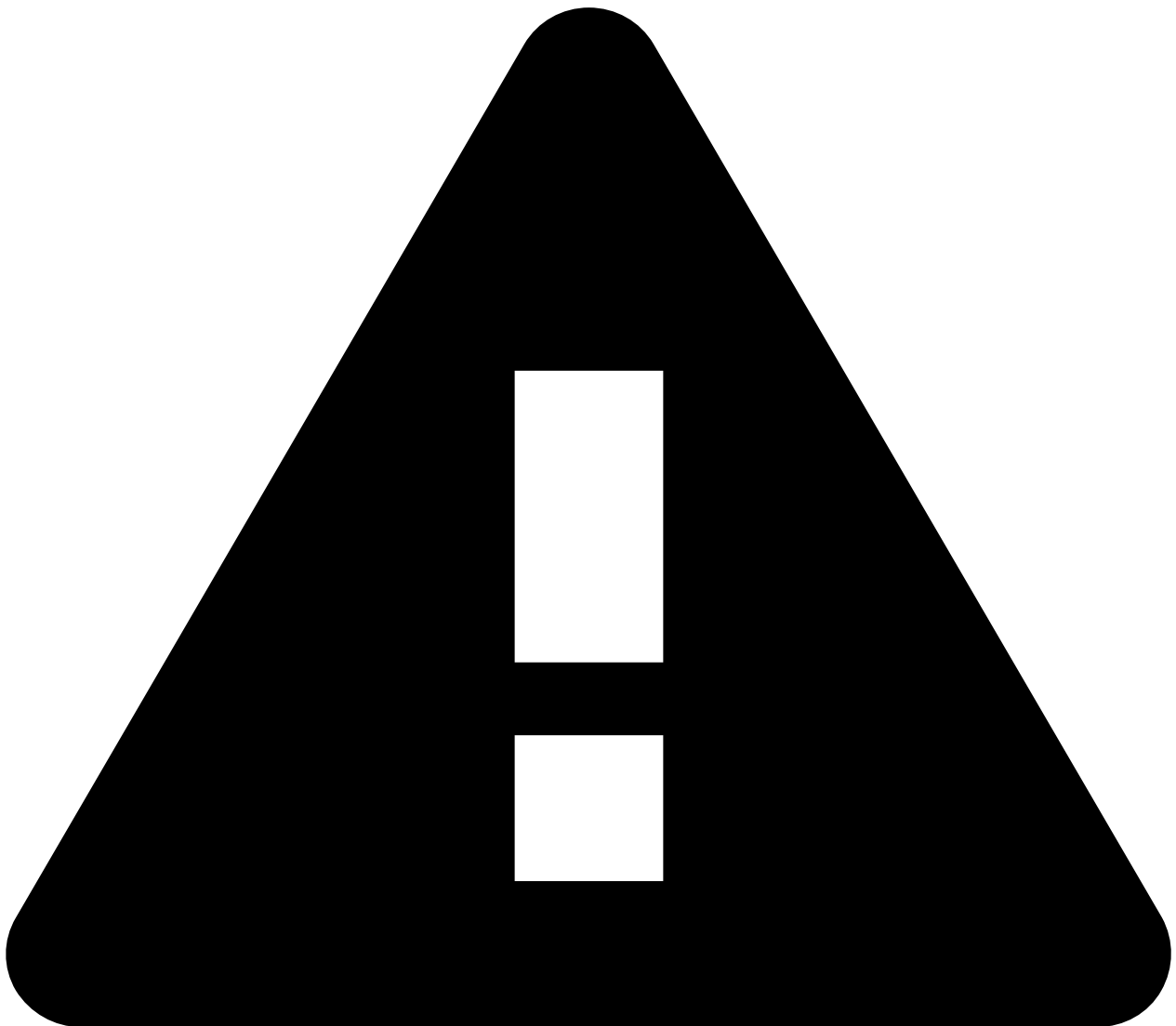
```
ipAddressSet(ipSubnets: Bag<IpSubnet>) : Set<IpAddress>
```

Returns a set of IP addresses from a collection of subnets, such as

```
ipAddressSet([ipSubnet("1.2.3.4/24"), ipSubnet("5.6.7.8/24")])
```

```
or ipAddressSet([ipSubnet("1234::/64"), ipSubnet("abcd::/64")]).
```

The space of IPv4 addresses is disjoint from the space of IPv6 addresses.



warning

A runtime error will occur if more than 20,000 subnets are required to express either the IPv4 addresses or the IPv6 addresses and the set is returned from a query or passed to [toString](#) or [toStringTruncate](#).

A bag of [IpSubnet](#) s is different than a set of [IpAddress](#) es. For example, if

```
subnets = bag([ipSubnet("1.2.3.0/32"), ipSubnet("1.2.3.1/32")])
```

, then

- `toString(subnets)` returns `"[1.2.3.0/32, 1.2.3.1/32]"` but

- `toString(ipAddressSet(subnets))` returns `"[1.2.3.0/31]"`.

See also[â](#)

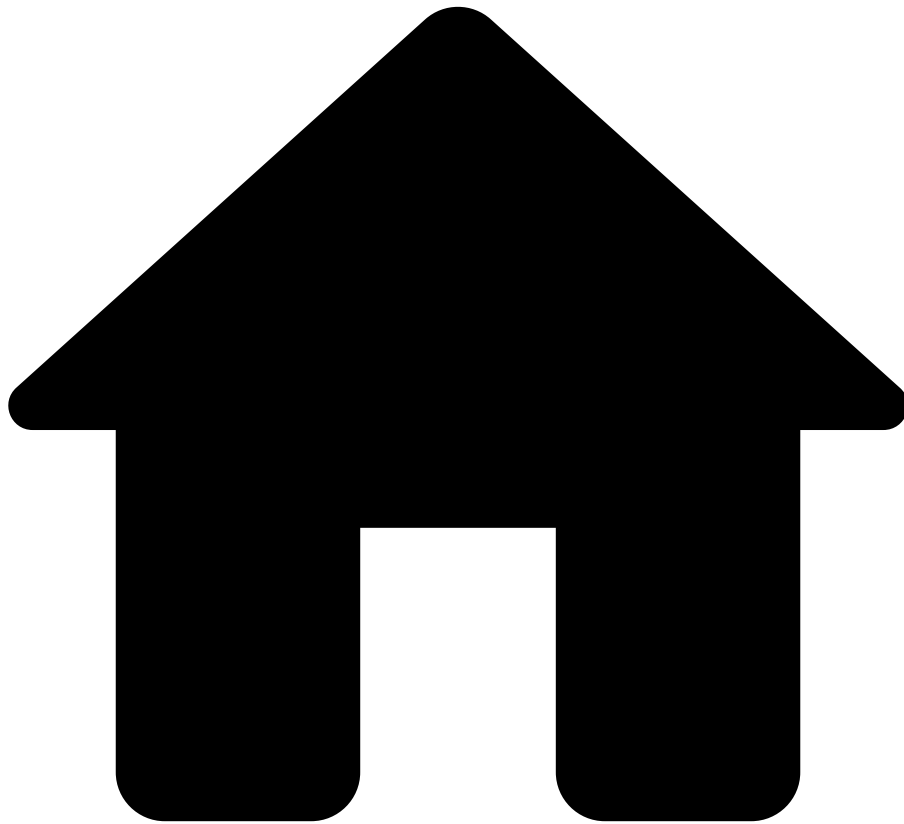
Types[â](#)

- [IpSubnet](#)
- [Collections](#)
- [Set](#)
- [IpAddress](#)

Functions[â](#)

- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipSubnet

On this page

ipSubnet

```
ipSubnet(str: String) : IpSubnet
```

Returns a subnet from a string, such as `ipSubnet("1.2.3.4/24")`.

```
ipSubnet(ipAddr: Ipv4Address, prefixLength: Integer): IpSubnet
```

Returns a subnet from an IP address and length, such as

```
ipSubnet(ipAddress("1.2.3.4"), 24) .
```

```
ipSubnet(ipAddr: Ipv4Address, netMask: Ipv4Address): IpSubnet
```

Returns a subnet from an IP address and a net mask represented as an IP address, such as `ipSubnet(ipAddress("1.2.3.4"), ipAddress("255.255.128.0"))`, which is equivalent to `ipSubnet("1.2.3.4/17")`. This function will raise an error if the net mask parameter (second parameter) is not a valid subnet mask (an IP address that consists of some number of leading 1 bits followed by all 0 bits).

See also[â€Ž](#)

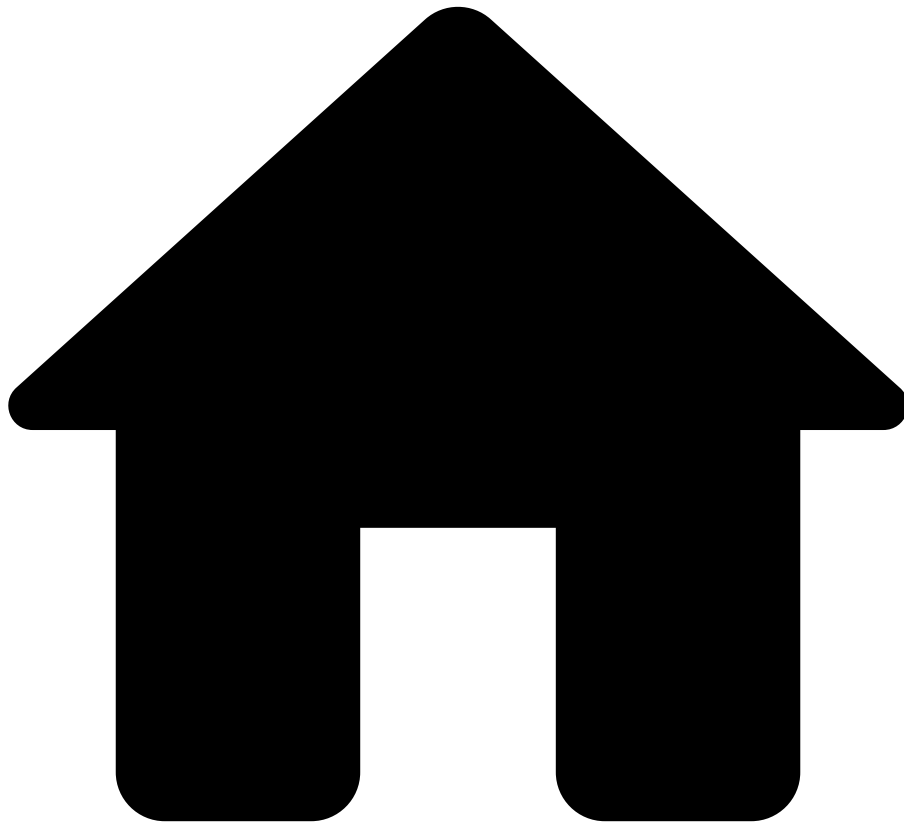
Types[â€Ž](#)

- [String](#)
- [IpSubnet](#)

Functions[â€Ž](#)

- [ipAddress](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipv4Address

On this page

ipv4Address

```
ipv4Address(num: Integer) : IPAddress
```

Constructs an IPv4 address from an integer in the range `0` to `(2 ^ 32) - 1`. For example,

1. `ipv4Address(0)` denotes `0.0.0.0`
2. `ipv4Address(4)` denotes `0.0.0.4`
3. `ipv4Address(1234)` denotes `0.0.4.210`
4. `ipv4Address(123456789)` denotes `7.91.205.21`
5. `ipv4Address((2 ^ 32) - 1)` denotes `255.255.255.255`

For numbers outside the range `0..2^32-1`, `ipv4Address` throws an error.

Whereas `ipv4Address` constructs IP addresses from *integers*, there is the [ipAddress](#) method to construct IP addresses from *strings*. Moreover, we can construct integers from IP addresses using [toNumber](#).

Properties

The [ipv4Address](#) function and the [toNumber](#) are inverses: `toNumber(ip) == num` is true exactly when `ip == ipv4Address(num)` is true, for `ip` an IPv4 address and `num` an integer in `0..2^32-1`. For example:

- `ipv4Address(toNumber(ipAddress("1.2.3.4"))) == ipAddress("1.2.3.4")`
- `toNumber(ipv4Address(1234)) == 1234`

See also

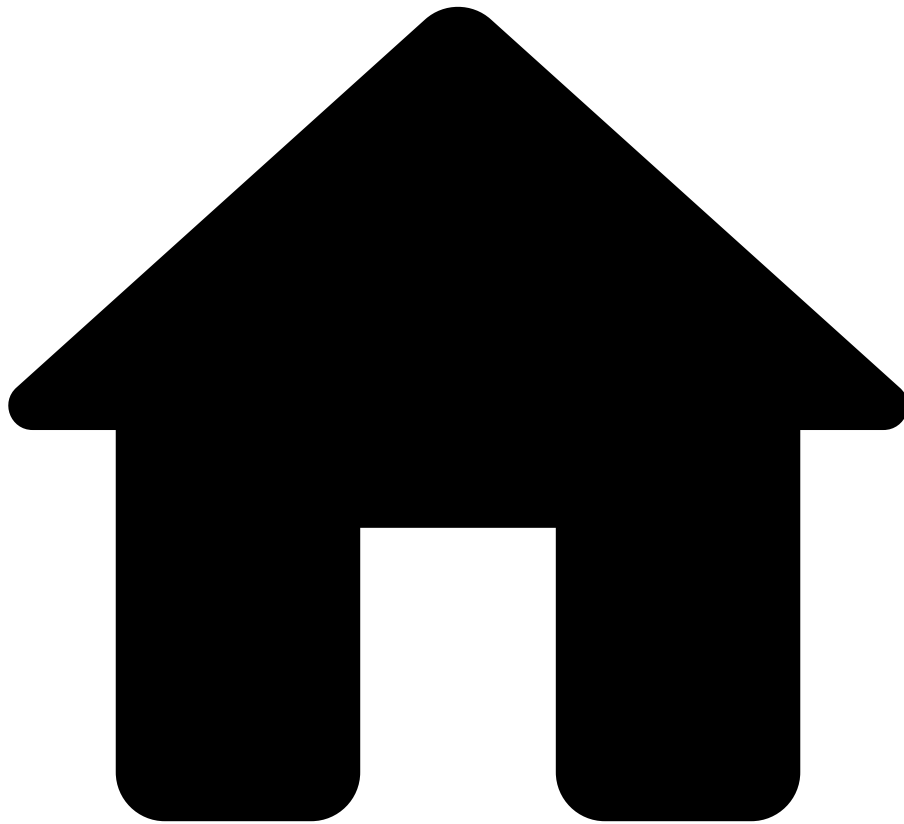
Types

- [ipAddress](#)
- [String](#)

Functions

- [toNumber](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isEmpty

On this page

isEmpty

```
isEmpty(set: Set<IpAddress>) : Bool
```

Returns a `Bool` indicating if `set` contains no elements.

For example,

- `isEmpty(ipAddressSet([ipSubnet("1.2.3.4/32")]))` returns `false` while
- `isEmpty(intersect(ipAddressSet([ipSubnet("1.2.3.4/32")]), ipAddressSet([ipSubnet("5.6.7.8/32")])))` returns `true`.

`isEmpty(str: String) : Bool`

Returns a `Bool` indicating if `str` has no characters. For example, `isEmpty("")` returns `true` while `isEmpty("hello")` returns `false`.

`isEmpty(collection: Bag<T>) : Bool` for any type `T`.

Returns a `Bool` indicating if a collection contains no elements.

- Note that this method accepts both `Lists` and `Bags`, since [Lists subtype Bags](#).

For example, `isEmpty([1, 2, 3])` returns `false` while `isEmpty(foreach x in fromTo(1, 0) select x)` returns `true`.

See also

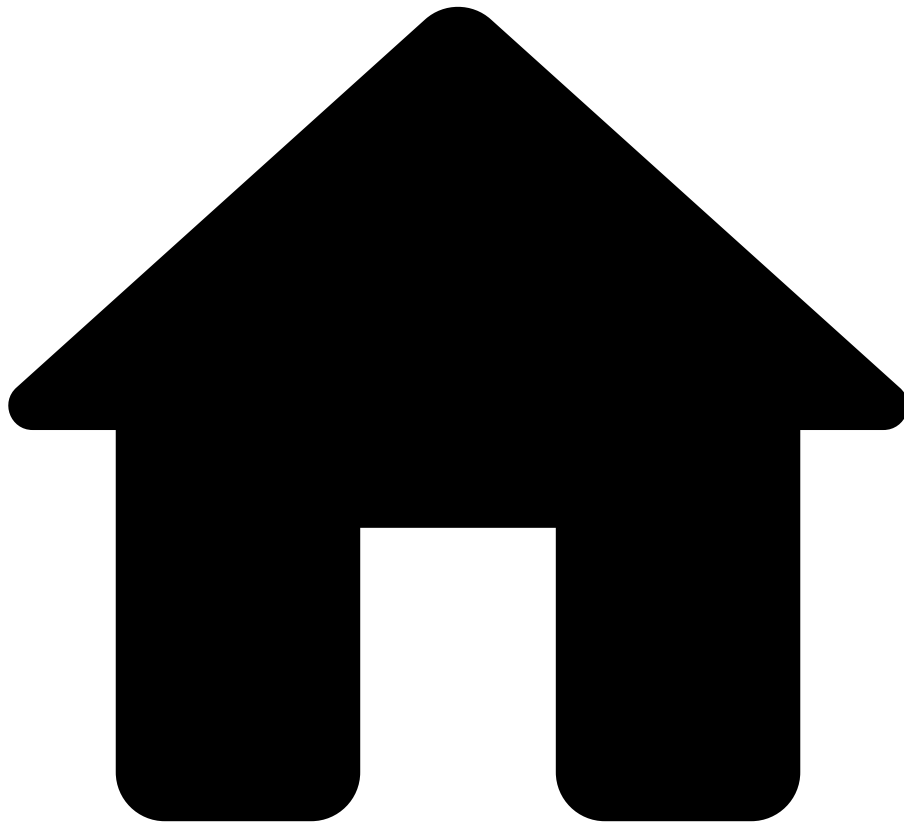
Types

- [Set](#)
- [Collections](#)
- [String](#)
- [Bool](#)

Functions

- [ipAddressSet](#)
- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isIPv4

On this page

isIPv4

```
isIPv4(address: IPAddress) : Bool
```

Tests whether the specified `IPAddress` is an IPv4 address. For example,

```
isIPv4(ipAddress("1.2.3.4")) returns true and
```

```
isIPv4(ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:7334"))  
returns false .
```

This function will raise an error if `address` is absent.

```
isIPv4(subnet: IpSubnet) : Bool
```

Tests whether the specified `IpSubnet` is an IPv4 subnet. For example,

```
isIPv4(ipSubnet("192.168.1.0/24")) returns true and  
isIPv4(ipSubnet("2001:0db8:abcd:0000::/56")) returns false .
```

This function will raise an error if `subnet` is absent.

An example usage would be the following query that gets all used IPv4 VPC subnets:

```
getVpcIPv4Subnets =  
  foreach cloudAccount in network.cloudAccounts  
  foreach vpc in cloudAccount  
  foreach subnet in vpc.subnets  
  foreach address in subnet.addresses  
  where isIPv4(address)  
  select address;
```

See also[â€Ž](#)

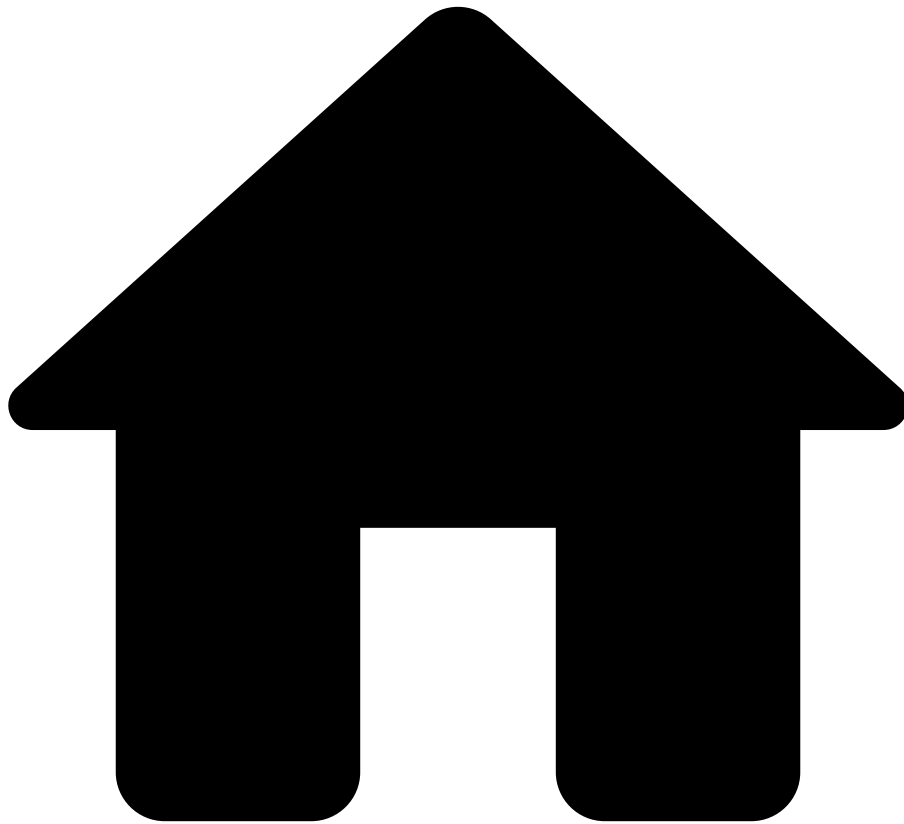
Types[â€Ž](#)

- [IpAddress](#)
- [IpSubnet](#)
- [Bool](#)

Functions[â€Ž](#)

- [ipAddress](#)
- [ipSubnet](#)
- [isIPv6](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isIPv6

On this page

isIPv6

`isIPv6(address: IPAddress) : Bool`

Tests whether the specified `IPAddress` is an IPv6 address. For example,

`isIPv6(ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:7334"))` returns `true` and `isIPv6(ipAddress("1.2.3.4"))` returns `false`.

This function will raise an error if `address` is absent.

```
isIPv6(subnet: IpSubnet) : Bool
```

Tests whether the specified `IpSubnet` is an IPv6 subnet. For example, `isIPv6(ipSubnet("2001:0db8:abcd:0000::/56"))` returns `true` and `isIPv6(ipSubnet("192.168.1.0/24"))` returns `false`.

This function will raise an error if `subnet` is absent.

An example usage would be the following query that gets all used IPv6 VPC subnets:

```
getVpcIPv6Subnets =  
  foreach cloudAccount in network.cloudAccounts  
  foreach vpc in cloudAccount  
  foreach subnet in vpc.subnets  
  foreach address in subnet.addresses  
  where isIPv6(address)  
  select address;
```

See also

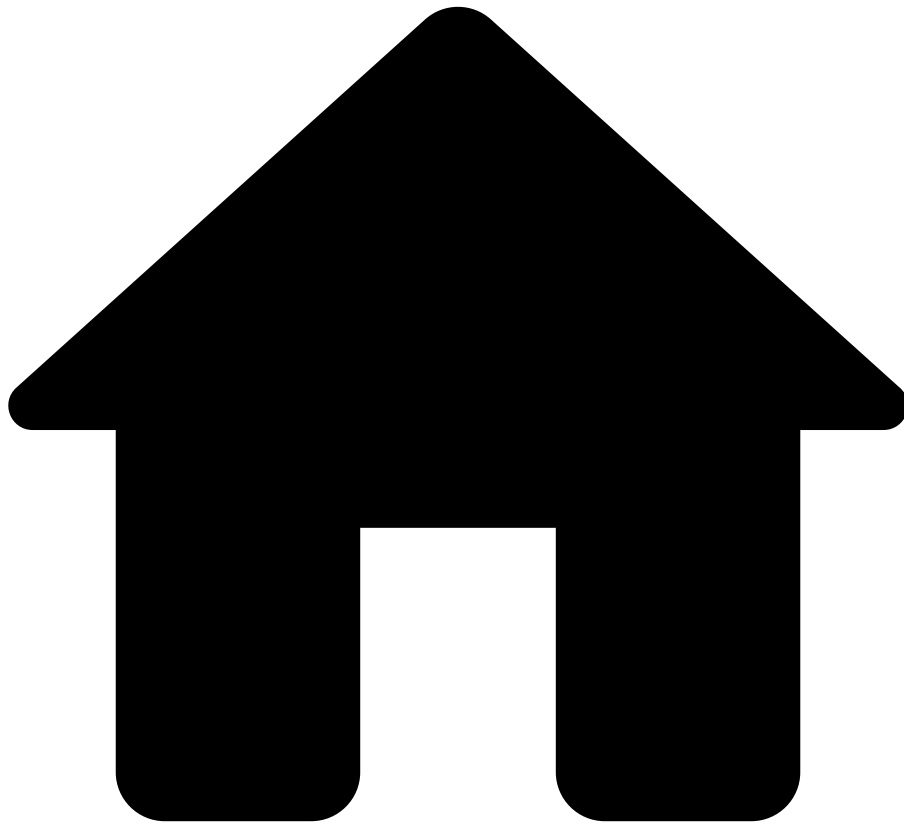
Types

- [IpAddress](#)
- [IpSubnet](#)
- [Bool](#)

Functions

- [ipAddress](#)
- [ipSubnet](#)
- [isIPv4](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isPresent

On this page

isPresent

`isPresent(value: T) : Bool` for any type `T`.

Returns `true` if a value is non-`null` and `false` otherwise. (Values like `[]`, `""`, `0` and `false` are not equal to `null` and return `true`.)

Examples:

- `isPresent("")` returns `true`
- `isPresent(0)` returns `true`
- `isPresent(null: Integer)` returns `false`
- `foreach l in network.locations select {x: isPresent(l.city)}`
returns `true` for each city in `network.locations` that has a value, and
`false` if the city is undefined.

See also

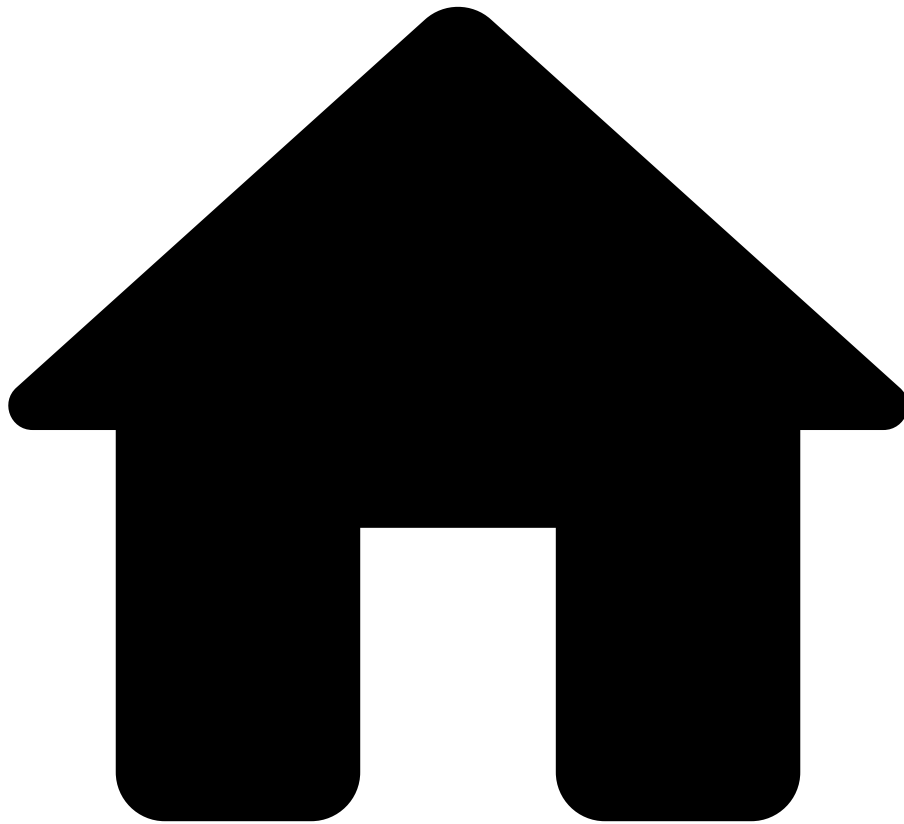
Functions

- [isEmpty](#)

Data Model

- [Location](#)

.



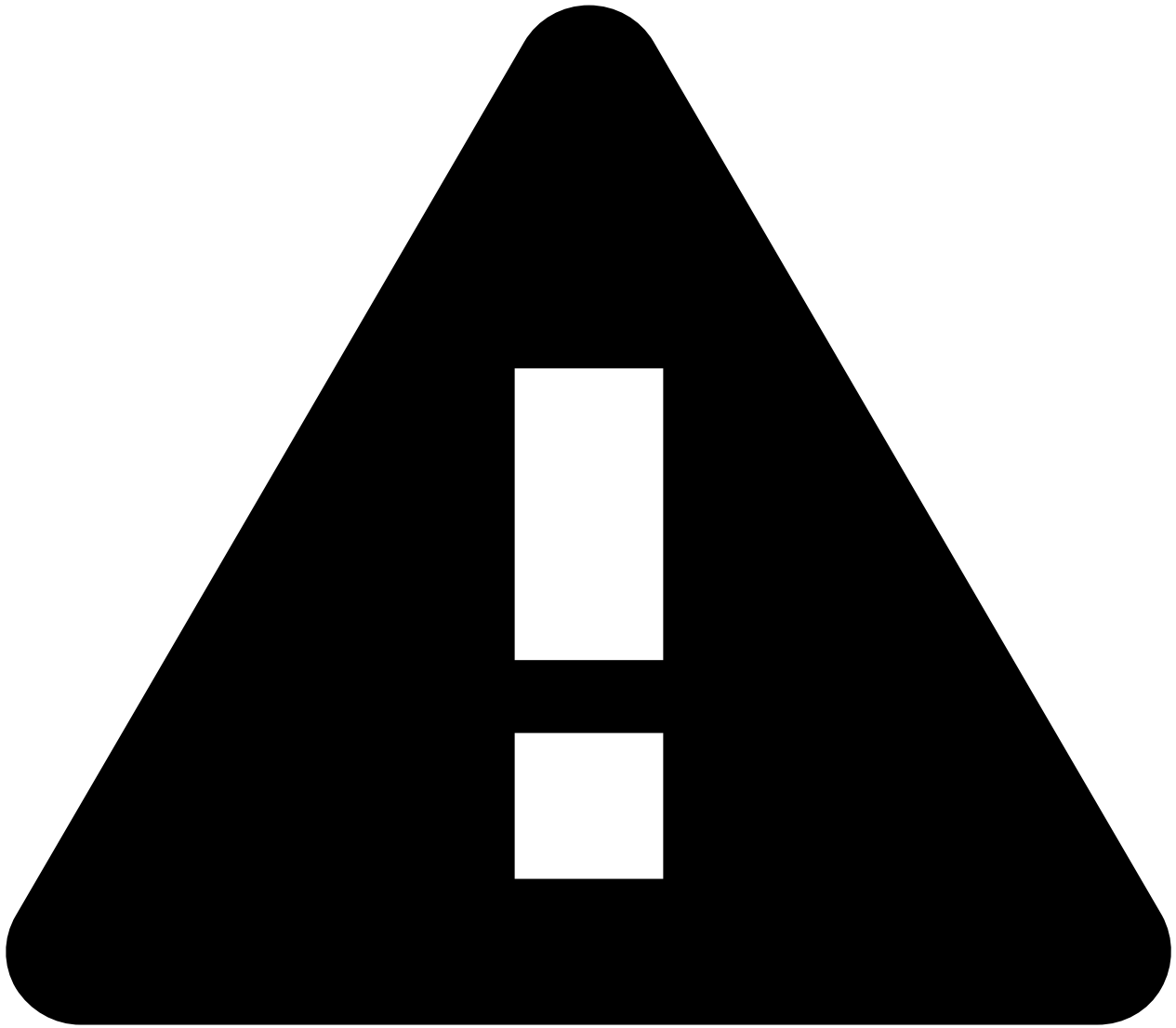
- Reference
- [NQE Language](#)
- [Standard Library](#)
- join

On this page

join

```
join(delimiter: String, list: List<String>) : String
```

Returns the result of concatenating together the elements in `list` of type `String` using `delimiter` between each element.



Deprecated Bag overload

As a transitional measure for backwards compatibility, `join` also accepts a `Bag<String>` as its second argument. Because Bags have non-deterministic ordering, the order of joined elements may be unpredictable. Convert the Bag to a List using `orderBy()` or `order()` to get deterministic results. In due time, the Bag overload will be removed and a type-checking error will be emitted instead.

Examples

```
join("", ["a", "b", "c"]) == "abc"
join(" ", ["a", "b", "c"]) == "a b c"
```

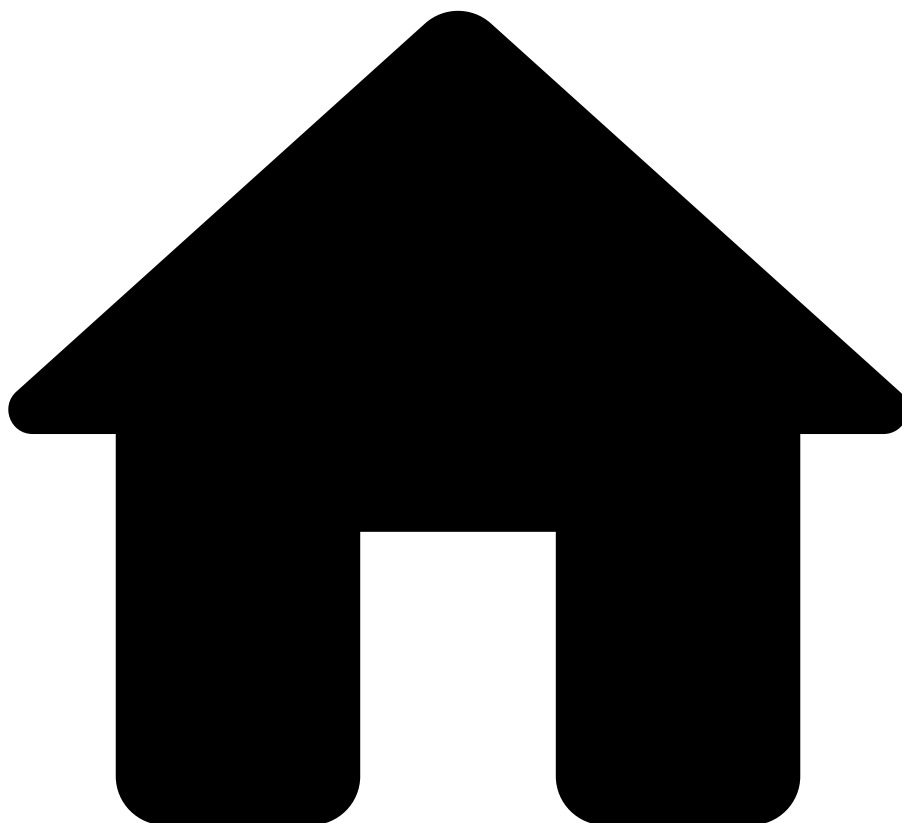
```
join(", ", ["a", "b", "c"])      == "a, b, c"  
join(" and ", ["a", "b", "c"])  == "a and b and c"
```

See also[â](#)

Types[â](#)

- [String](#)
- [Collections](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- length

On this page

length

For any type `T`,

```
length(collection: Bag<T>) : Integer
```

Returns the length of a collection. For example, `length(["a", "b"])` is `2`.

- Note that this method accepts both `Lists` and `Bags` —since [Lists subtype Bags](#)

```
length(str: String): Integer
```

Returns the number of characters in a string. For example, `length("abc")` is `3`.

```
length(subnet: IpSubnet): Integer
```

Returns the length of the subnet. For example,

```
length(ipSubnet("1.2.3.4/24")) is 24.
```

See also[â](#)

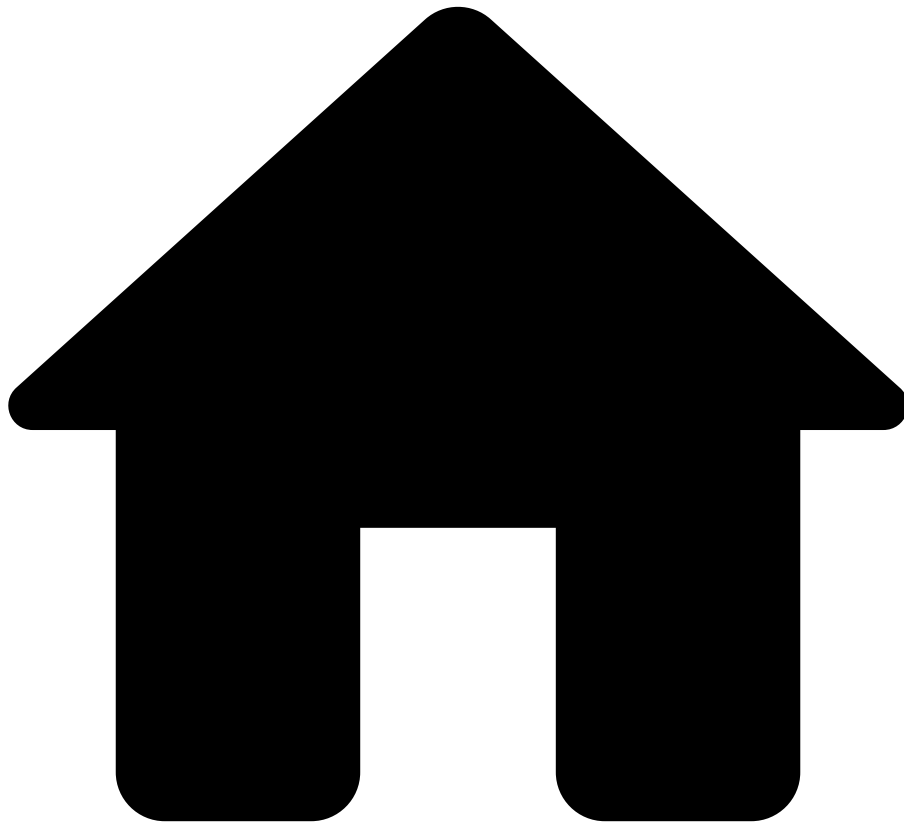
Types[â](#)

- [Collections](#)
- [Integer](#)
- [String](#)
- [IpSubnet](#)

Functions[â](#)

- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- limit

On this page

limit

```
limit(list: List<T>, n: Number) : List<T>
```

Gets the sublist of `list` consisting of the first `n` elements of `list`.

- The maximum number of elements to return, `n`, must be non-negative.

- If `n` is greater than the collection size, returns all elements.
- If `n` is 0, returns an empty collection

Examples

Basic Usage

```
numbers      = [1, 2, 3, 4, 5];  
firstThree   = limit(numbers, 3); // [1, 2, 3]
```

With order by Clause

```
limit(  
  foreach device in network.devices  
  select { Name: device.name }  
  order by Name desc,  
  10  
)
```

Using Comprehension Syntax

The `limit` clause in a comprehension provides a more readable alternative:

```
foreach device in network.devices  
select { Name: device.name }  
order by Name desc  
limit 10
```

See Also

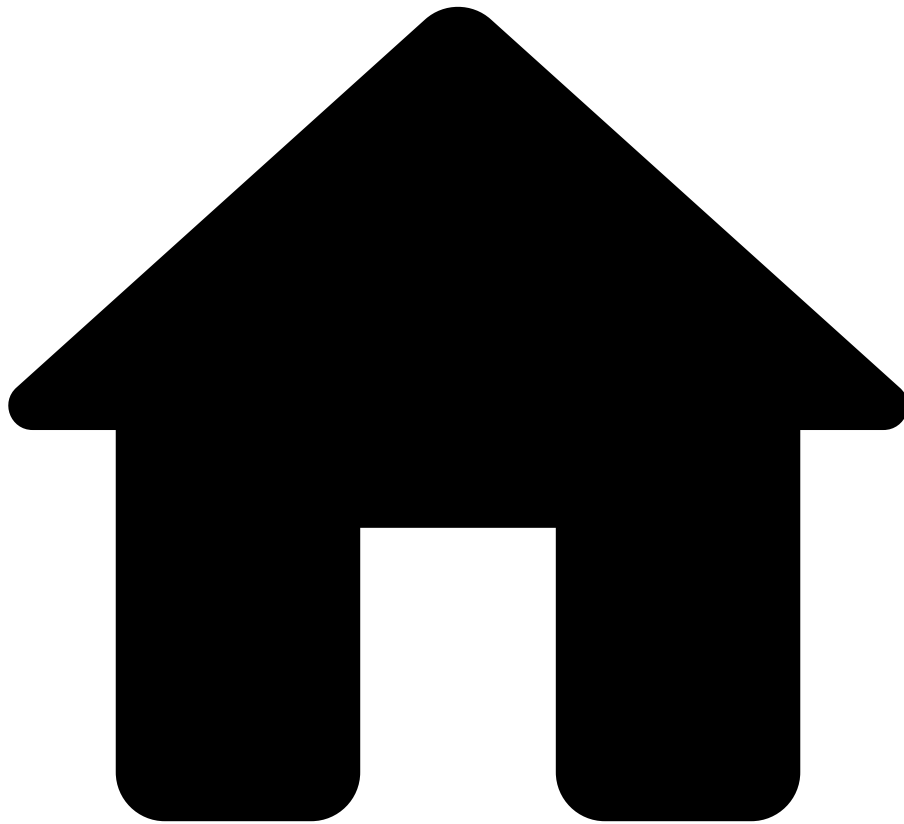
Guides

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering
- [Comprehensions](#) - Using limit clause

Functions [↗](#)

- [orderBy](#) - Sort a collection by a key function
- [length](#) - Get the length of a collection

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `macAddress`

On this page

macAddress

```
macAddress(str: String) : MacAddress
```

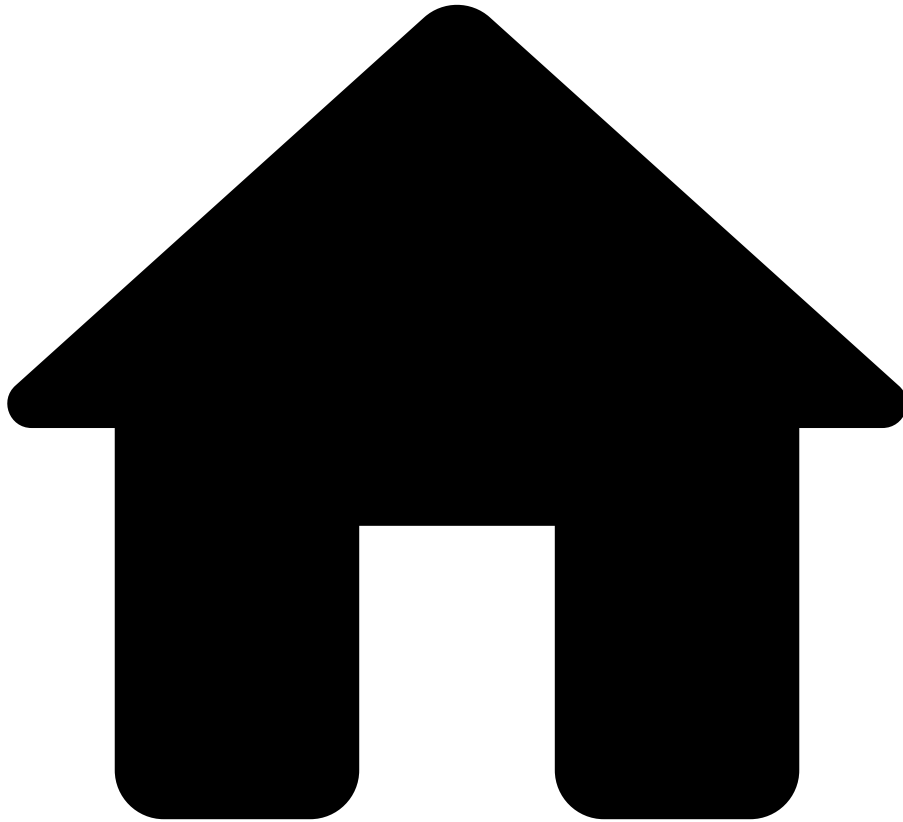
MAC addresses can be written like this: `macAddress("11:22:33:44:55:66")`.

See also [ââ](#)

Types [ââ](#)

- [MacAddress](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- match

On this page

match

```
match(text: String, regex: Regex<T>) : T?
```

Gets a record of captured data if `text` matches the structure specified by `regex`, or otherwise `null`.

See [the regular expression guide](#).

Examples

Example	text	regex
1	"Jane is 32 years old"	re`Jane is 32 years old`
2	"Jane is 33 years old"	re`Jane is 32 years old`
3	"Name: Jane, Age: 32"	re`Name: (\w+), Age: (\d+)`
4	"Name: Jane, Age: 32"	re`Name: (?<name>\w+), Age: (?<age:Integer>\d+)`
5	"Jane is 32 years old"	re`(?<age:Integer>\d+) (?: (?<name>\w+))`
6	"32"	re`(?<age:Integer>\d+) (?: (?<name>\w+))`
7	"3x2"	re`(?<age:Integer>\d+) (?: (?<name>\w+))`

Regexes with anonymous captures have match result record with indices as keys, and all values are of type `String`.

- Matches of *unnamed* capture groups are referenced with string indices, such as `"1"` and not numbers such as `1`.
- For instance, in example 3 above, the regex has type `Regex<{"1": String, "2": String}>`. As such, `match("Name: Jane, Age: 32", re`Name: (\w+), Age: (\d+)`)[ "2" ]` is how we access the second capture group, which results in the `String` value `"32"`.

Regexes with *named* captures have match result record with the declared names as keys, and all values are of declared types

- For instance, in example 4 above, the regex has type `Regex<{name: String, age: Integer}>`, and so `match("Name: Jane, Age: 32", re`Name: (?<name>\w+), Age: (?<age:Integer>\d+)`).age` is how we access the capture group *named* `age`, resulting in the `Integer` value `32`.
- Moreover, notice that the capture group `name` has no declared type, so its type defaults to `String`.

See also[â](#)

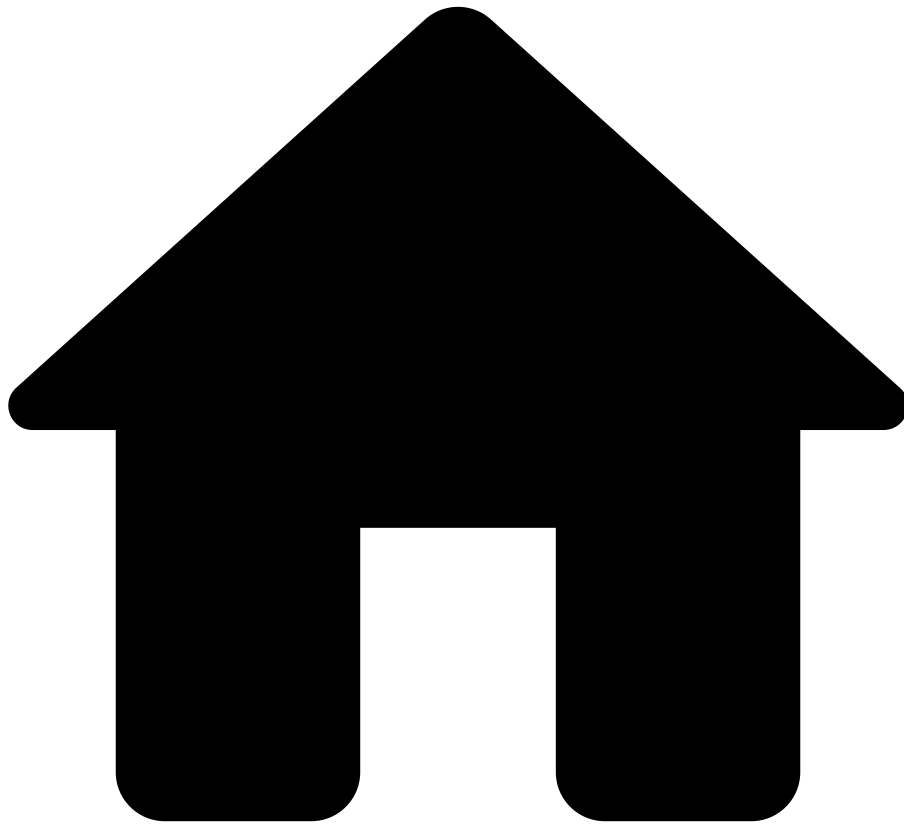
Functions[â](#)

- [hasMatch](#): Checks whether a string matches the structure specified by a regex or not.
- [regexMatches](#): Gets the list of all substring matches of a given regex.

Types[â](#)

See [the regular expression guide](#).

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- matches

On this page

matches

```
matches(str: String, glob: String) : Bool
```

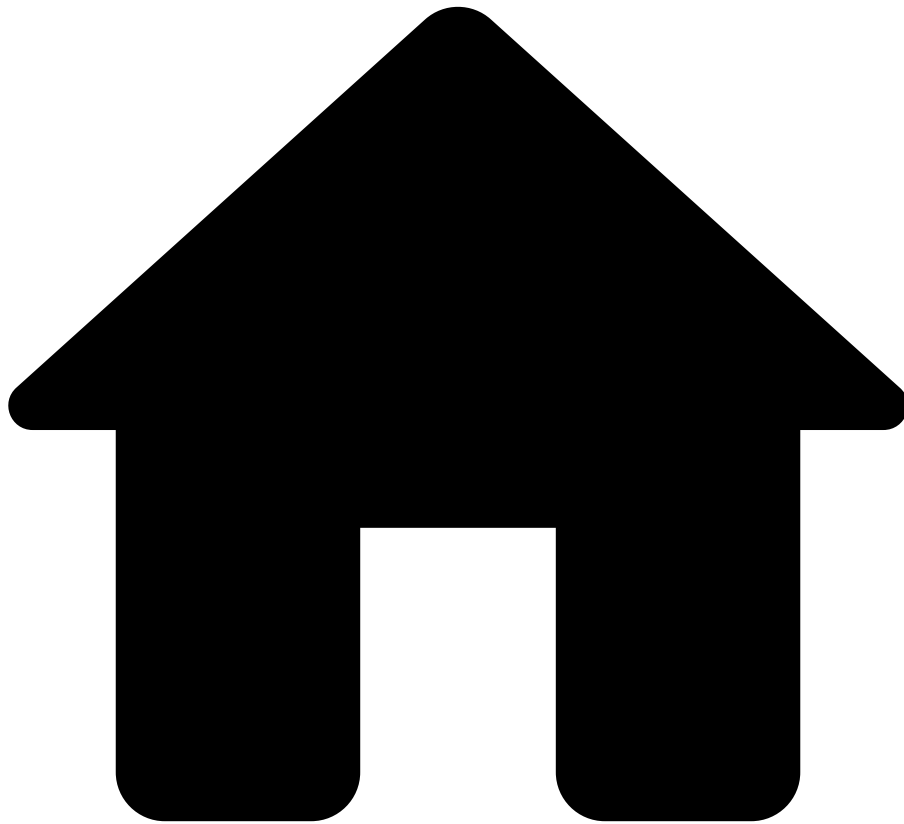
Returns `true` if and only if `str` matches `glob`, case-sensitively. For example, `matches("def", "def")` and `matches("abcdefefg", "*def*")` both return `true`.

See also [a](#)

Types [a](#)

- [String](#)
- [Bool](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- max

On this page

max

For any type `T`,

```
max(collection: Bag<T>) : T?
```

Returns the greatest item in the `collection`, or `null` if it is empty.

- The greatest item in a collection is selected with the same order used for comparing items with relational expressions such as less-than (`<`) and greater-than (`>`).
- See how [comparisons](#) work for more detail on the order of values.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

See also[â](#)

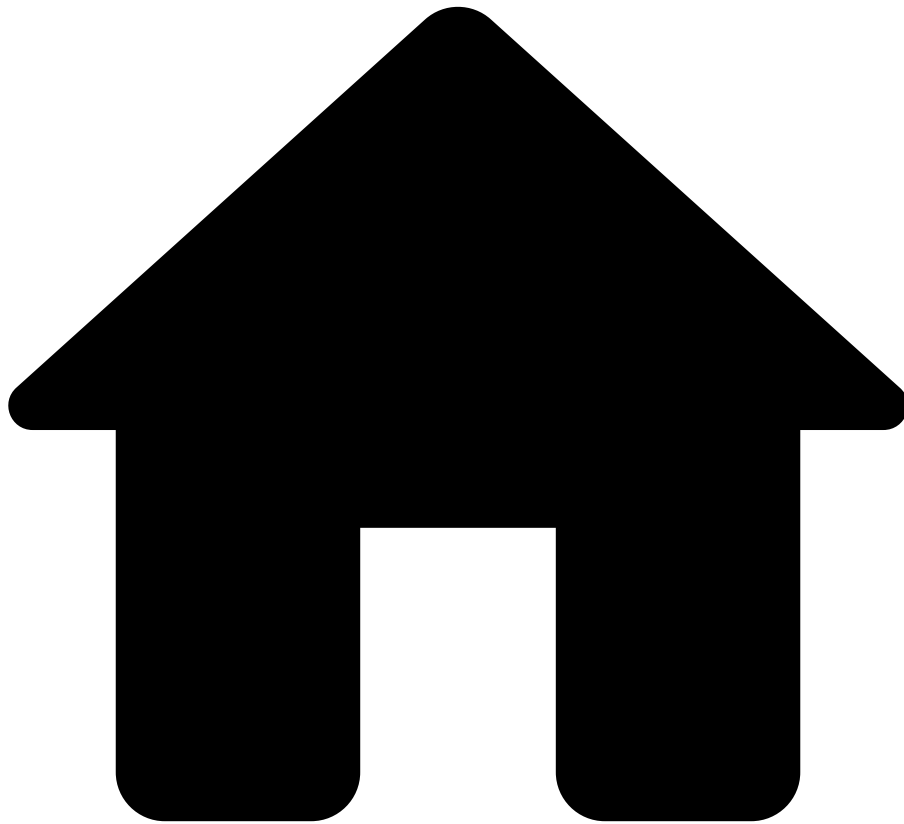
Types[â](#)

- [Collections](#)

Functions[â](#)

- [maxBy](#)
- [min](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `maxBy`

On this page

maxBy

For any types `T` and `K`,

```
maxBy(collection: Bag<T>, keySelector: T -> K) : T?
```

Gets the greatest item in the `collection`, where items are ordered by the value returned by applying `keySelector` to the item.

- For example, if the following function has been declared `getRoutePrefix(route) = route.prefix;`, and if `routes` is a variable referring to some list of `IpEntry` records, then `maxBy(routes, getRoutePrefix)` evaluates to the route with the largest prefix (aka subnet). See how [comparisons](#) work for more on how subnets are ordered.
- Similar to `max` and `min`, this function returns `null` if there are no items in the collection.
- Note that this method accepts both `Lists` and `Bags` —since [Lists subtype Bags](#)

See also

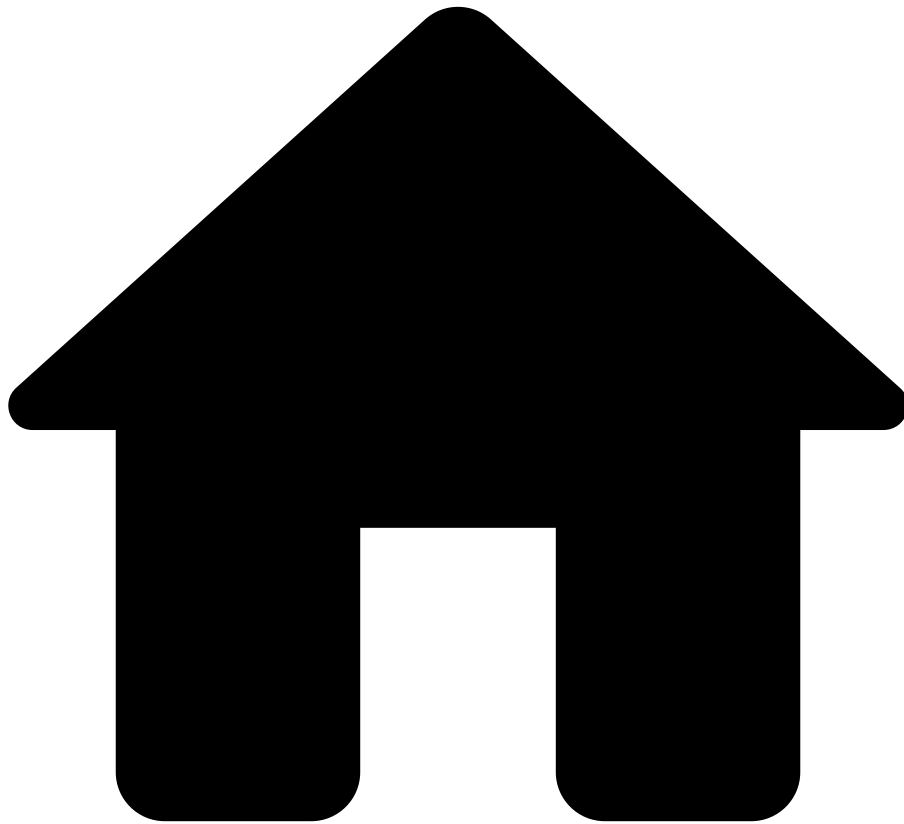
Types

- [Collections](#)

Functions

- [max](#)
- [minBy](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- mergeBlocks

mergeBlocks

```
mergeBlocks(blockPattern: PatternBlocks<{}>) : PatternBlocks<{}>
```

`mergeBlocks(blockPattern)` combines the patterns and sub-patterns in `blockPattern` based on common roots. This can be useful if the pattern contains multiple sub-patterns that each specify part of the desired block properties. For example, consider the following `pattern1` :

```

pattern1 = ```
interface GigabitEthernet1/0/1
  switchport mode access
  hsrp 30
    preempt
router bgp
interface GigabitEthernet1/0/1
  description Important port
  hsrp 30
    priority 50
```;

```

The value of `mergeBlocks(pattern1)` will be the following pattern, `pattern2`, which has all sub-blocks of the interface combined into one block:

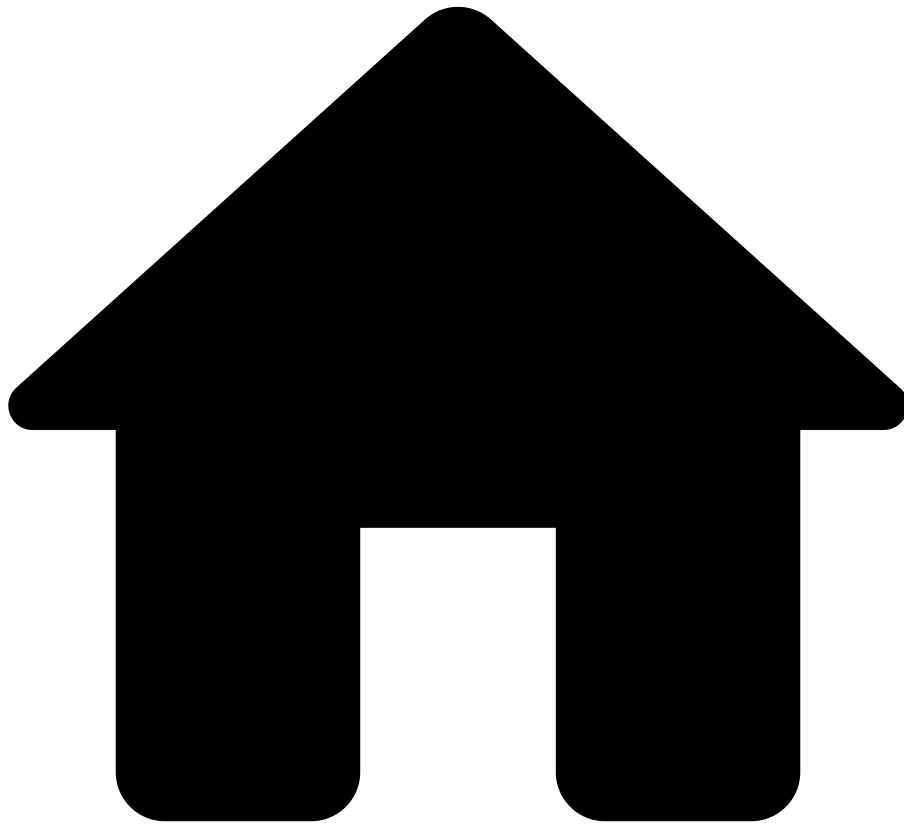
```

pattern2 = ```
interface GigabitEthernet1/0/1
 switchport mode access
 hsrp 30
 preempt
 priority 50
 description Important port
router bgp
```;

```

Note that `mergeBlocks(pattern)` works at all nesting levels of a block pattern. In other words, it will merge blocks not just at the root level, but also will apply block merging for the merged subtrees of a block. This can be seen in the above example, where the multiple `hsrp 30` have been merged together into a single block.

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- min

On this page

min

For any type `T`,

```
min(collection: Bag<T>) : T?
```

Gets the least item in the `collection`, or `null` if there are no items in the collection.

- The least item in a collection is selected with the same order used for comparing items with relational expressions such as less-than (`<`) and greater-than (`>`).
- See how [comparisons](#) work for more detail on the order of values.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

See also[â](#)

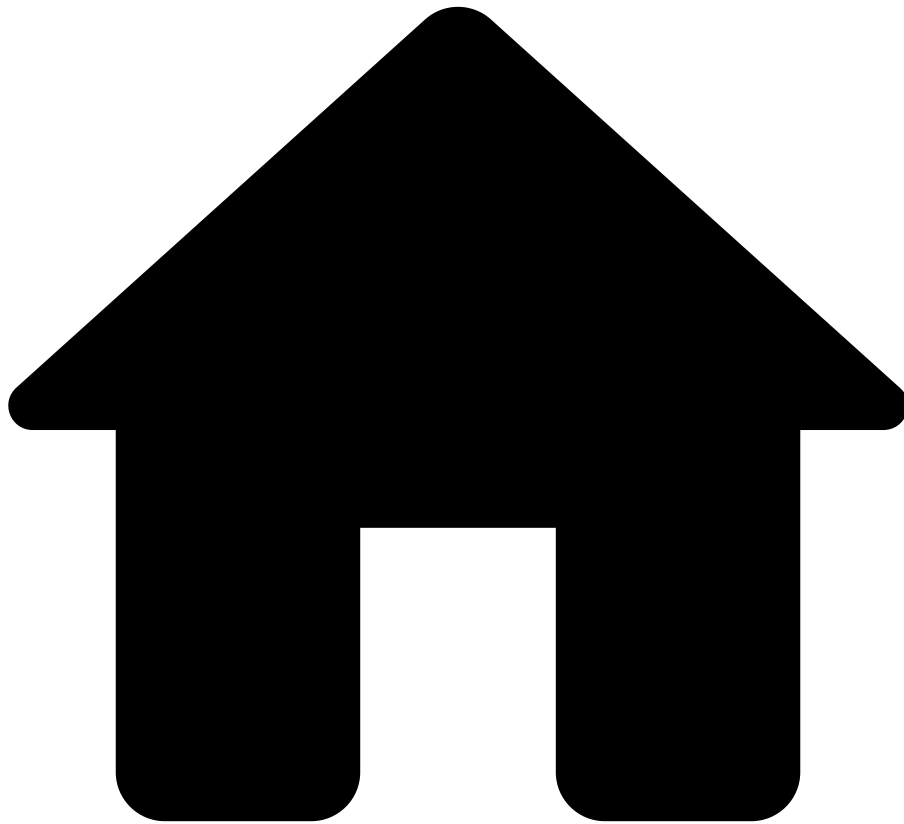
Types[â](#)

- [Collections](#)

Functions[â](#)

- [minBy](#)
- [max](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- minBy

On this page

minBy

For any types `T` and `K`,

```
minBy(collection: Bag<T>, keySelector: T -> K) : T?
```

Gets the least item in the `collection`, where items are ordered by the value returned by applying `selector` to each item.

- For example, if the following function has been declared `getRoutePrefix(route) = route.prefix;`, and if `routes` is a variable referring to some list of `IpEntry` records, then `minBy(routes, getRoutePrefix)` evaluates to the route with the smallest prefix (aka subnet). See how [comparisons](#) work for more on how subnets are ordered.
- Similar to `max` and `min`, this function returns `null` if there are no items in the collection.
- Note that this method accepts both `Lists` and `Bags` — since [Lists subtype Bags](#)

See also

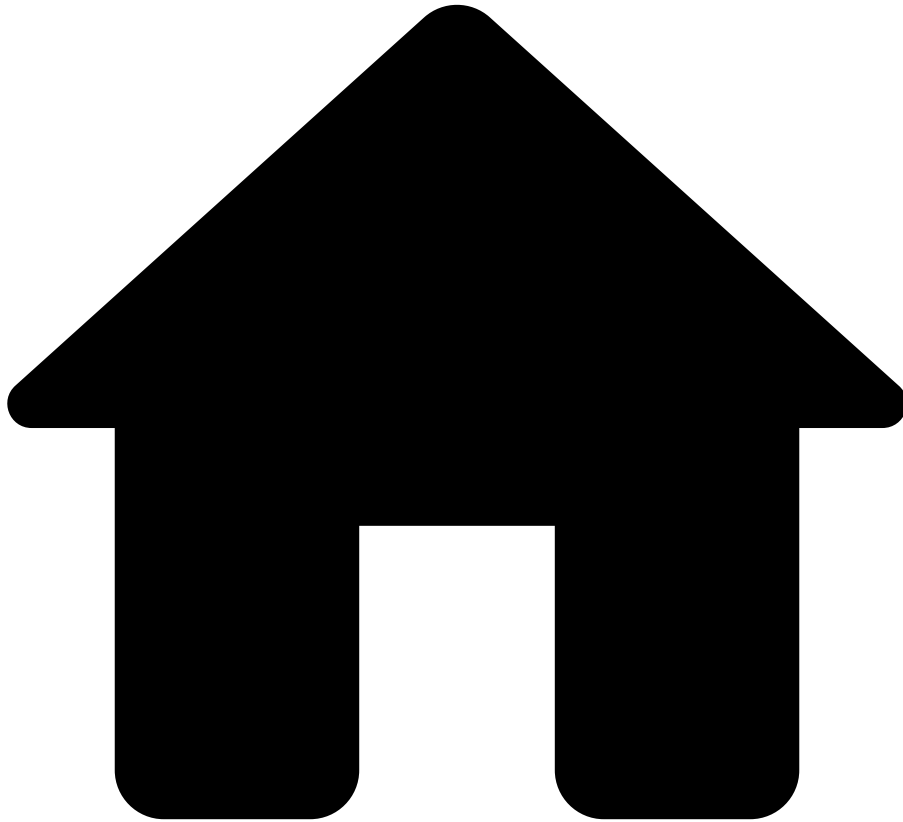
Types

- [Collections](#)

Functions

- [min](#)
- [maxBy](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- minutes

On this page

minutes

`minutes(value: Integer) : Duration`

Obtains a `Duration` representing the given number of minutes. For example, `minutes(3)` returns a `Duration` that represents three minutes. The value can be

negative. For example, `minutes(-2)` returns a `Duration` that represents negative two minutes.

This function will raise an error if the absolute value of the input is greater than `153722867280912930`.

See also

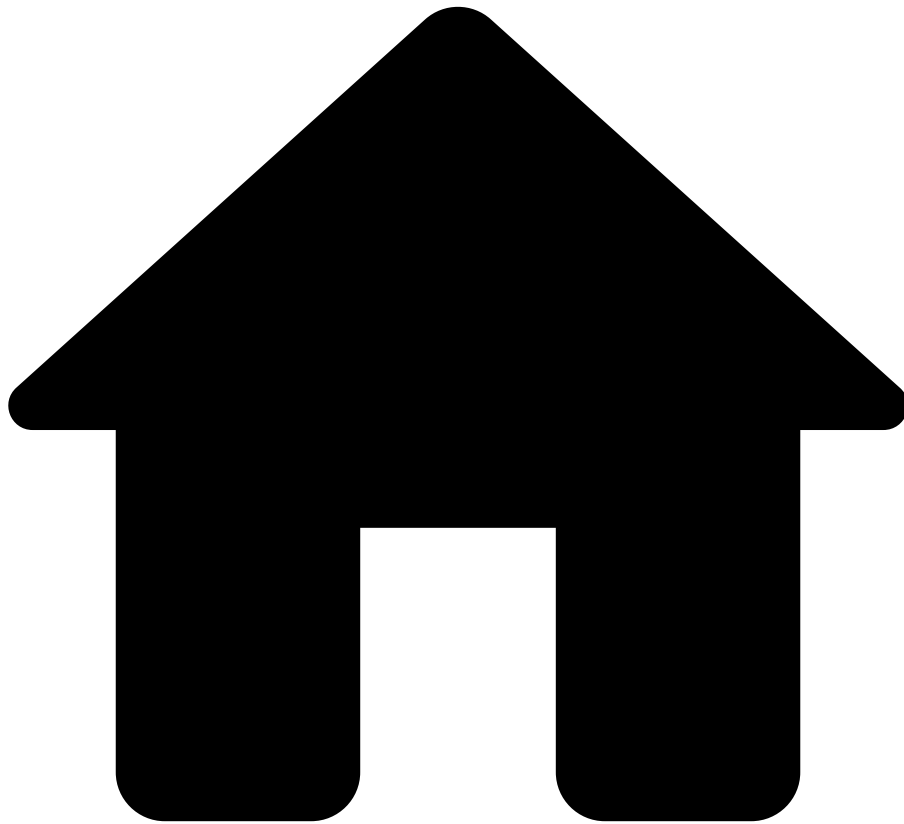
Functions

- [date](#)
- [timestamp](#)
- [duration](#)
- [days](#)
- [hours](#)
- [seconds](#)

Types

- [Time](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `networkAddress`

On this page

networkAddress

```
networkAddress(subnet: IpSubnet) : IpAddress
```

Returns the network IP address for the subnet, that is the IP address for the subnet, with all host bits set to 0. For example, `networkAddress(ipSubnet("1.2.3.4/24"))` is `ipAddress("1.2.3.0")`.

See also[â](#)

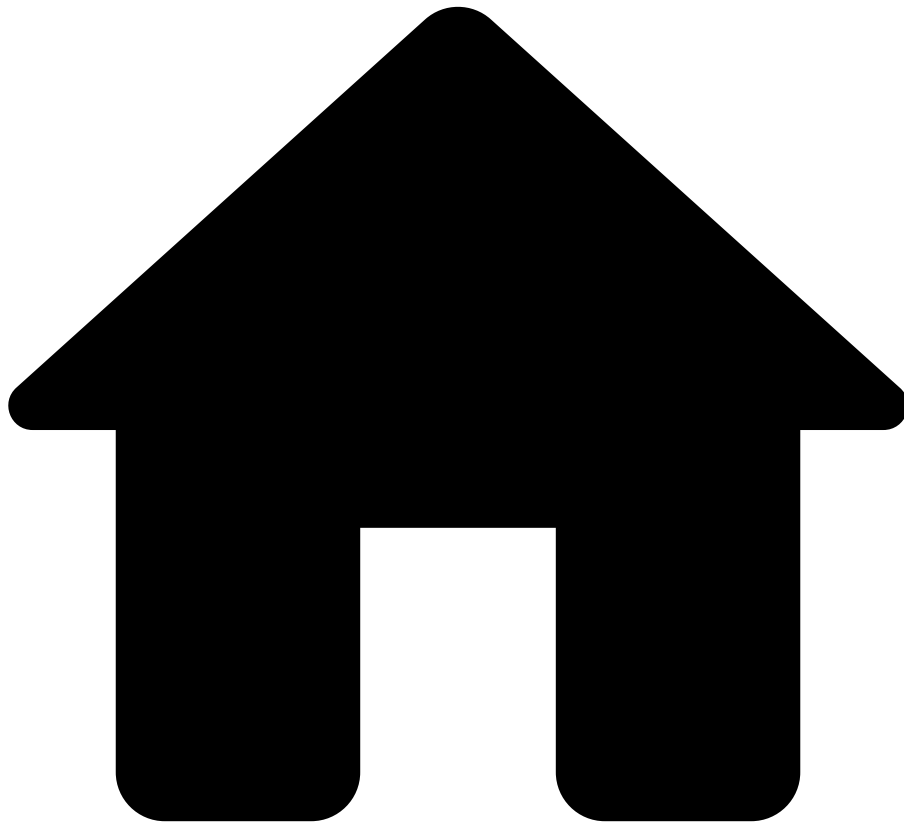
Types[â](#)

- [IpSubnet](#)
- [IpAddress](#)

Functions[â](#)

- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- order

On this page

order

For any type `T`,

```
order(collection: Bag<T>) : List<T>
```

Sorts a collection (in ascending order).

- The collection is ordered by the natural order on elements.
 - That is, the same order used for comparing items with relational expressions such as less-than (`<`) and greater-than (`>`).
 - See [comparisons](#) for more detail on the order of values.
- Note: `x <= y` if and only if `order([x, y])[0] == x` and `order([x, y])[1] == y`.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

Example

Sort a list of numbers in ascending order:

```
numbers = [3, 1, 4, 1, 5];  
order(numbers) // Returns [1, 1, 3, 4, 5]
```

This is equivalent to the more verbose, albeit more flexible, comprehension:

```
numbers = [3, 1, 4, 1, 5];  
  
foreach n in numbers  
select n  
order by n ascending
```

Notes

To sort in *descending order*, use the `order by` comprehension clause. See [collections](#) for more detail.

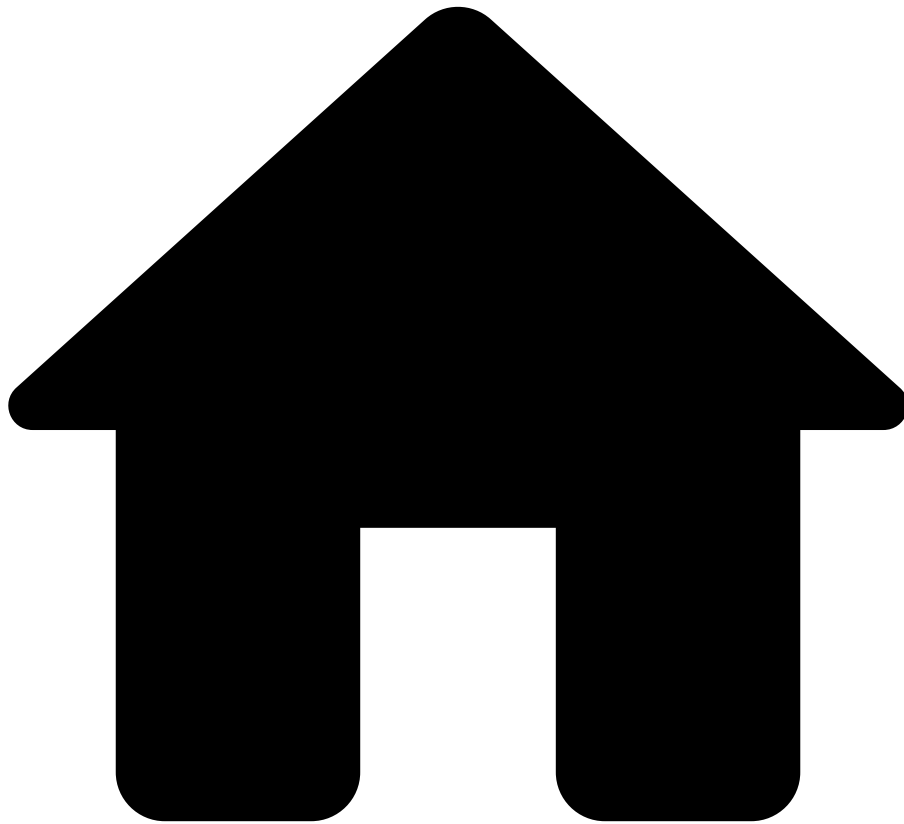
See Also

Guides

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering

- [Comprehensions](#) - Using order by clause

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- orderBy

On this page

orderBy

For any types `T` and `K`,

```
orderBy(collection: Bag<T>, keySelector: T -> K) : List<T>
```

Sorts a collection by applying a key selector function to each element and ordering by the resulting keys.

- The collection is sorted in ascending order based on the values returned by `keySelector`.
- For descending order, consider using the `order by` clause with `desc` modifier in a [comprehension](#), or negate numeric keys.
- See [comparisons](#) for more detail on the order of values.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

Example

Sorting Devices by Name

```
getName(device) = device.name;  
orderBy(network.devices, getName);
```

This is equivalent to the more verbose, albeit more flexible, comprehension:

```
foreach device in network.devices  
let name = device.name  
select device  
order by name ascending
```

Notes

- For sorting by multiple keys or for descending order, consider using the `order by` clause in a comprehension instead.
- Elements with equal keys maintain their relative order from the original list (stable sort).

See Also[â€‹](#)

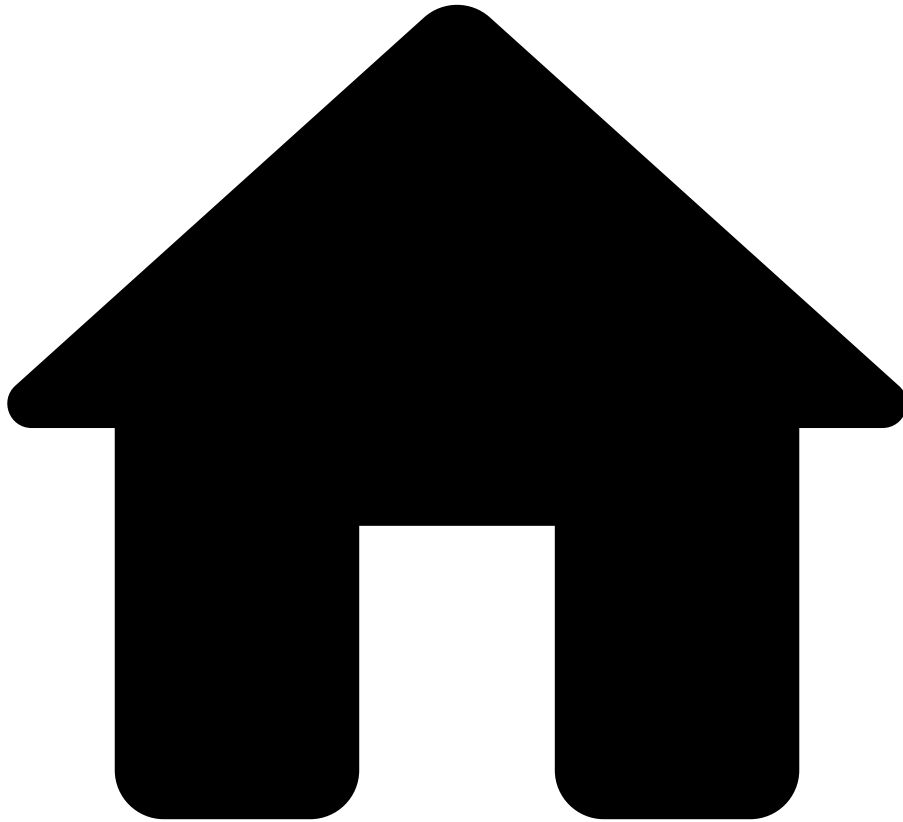
Guides[â€‹](#)

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering
- [Comprehensions](#) - Using order by clause

Functions[â€‹](#)

- [limit](#) - Limit the number of elements in a list
- [max](#) - Get the maximum element
- [min](#) - Get the minimum element
- [maxBy](#) - Get element with maximum key
- [minBy](#) - Get element with minimum key

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ouiAssignee

On this page

ouiAssignee

```
ouiAssignee(mac: MacAddress) : String?
```

Returns the name of the OUI assignee (as a `String`) associated with a MAC address. For example, `ouiAssignee(macAddress("00:55:DA:50:00:00"))` returns the

string `"Nanoleaf"` . The function can return a `null` value if the assignee for the MAC address is not known.

See also

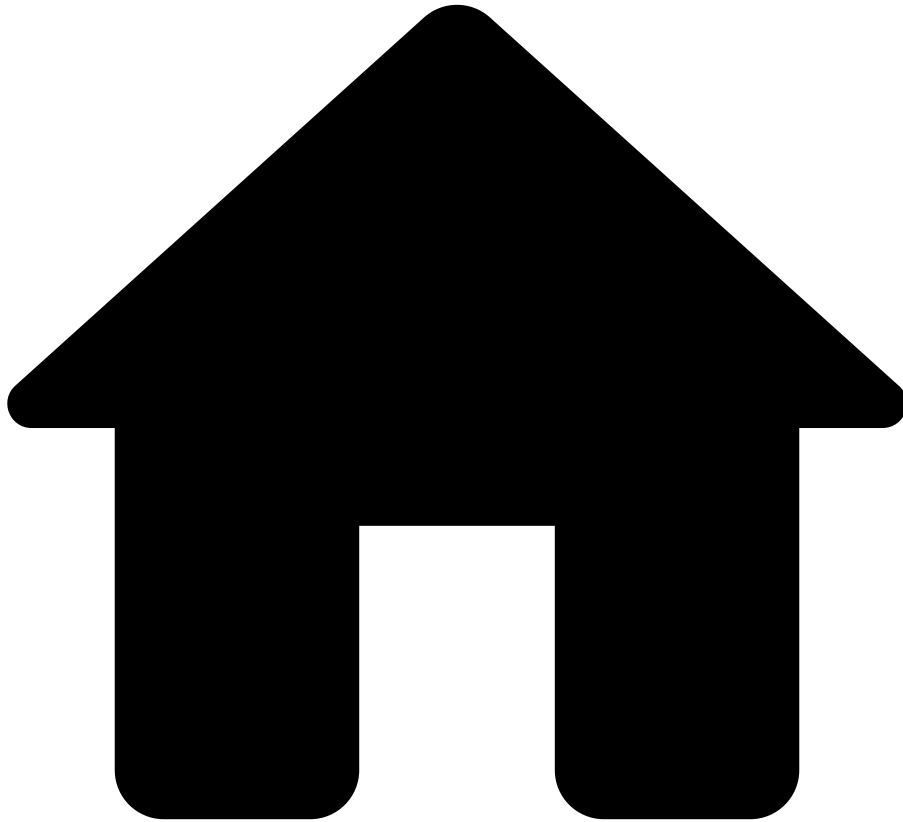
Types

- [MacAddress](#)
- [Collections](#)

Functions

- [ouiVendors](#)
- [macAddress](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ouiVendors

On this page

ouiVendors

```
ouiVendors(mac: MacAddress) : Bag<Vendor>
```

Returns a bag of `Vendor` s associated with a MAC address. For example,

```
ouiVendors(macAddress("00:05:00:00:00:0D")) returns  
bag([Vendor.CISCO]) and
```

```
ouiVendors(macAddress("F8:0D:AC:FF:FF:FF")) returns  
bag([Vendor.HP, Vendor.ARUBA]) .
```

See also

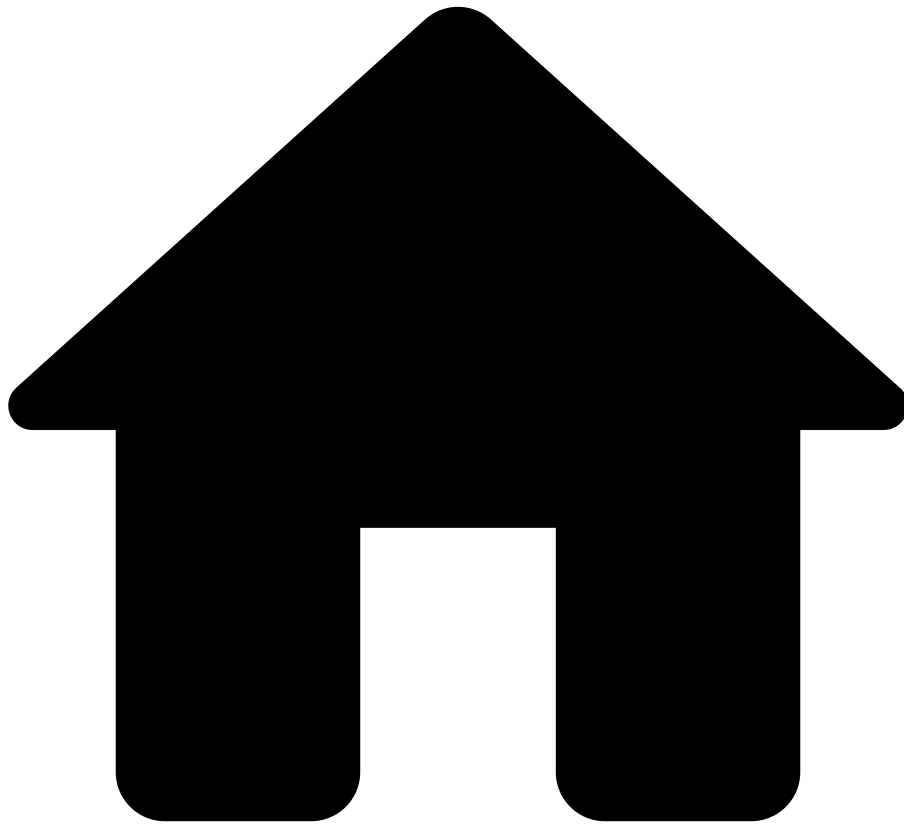
Types

- [MacAddress](#)
- [Collections](#)

Functions

- [ouiAssignee](#)
- [macAddress](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- parseConfigBlocks

On this page

parseConfigBlocks

```
parseConfigBlocks(os: OS, text: String) : List<ConfigLine>
```

The `parseConfigBlocks(os, text)` function can be used to parse a collection of configuration blocks from a string that holds a device configuration. In other words, an expression like `parseConfigBlocks(OS.NXOS, "interface eth1\n no`

`shut\n")` could be used to parse some NXOS configuration in the given string. The resulting value can then be used with various pattern-matching functions.

This function may be useful to process the response to custom commands, for example the response to command "show running-config all".

Not all lines in the given `text` correspond to some `ConfigLine` in the output. The lines that are ignored depend on the value of `os`. Here are all the ways that an input line is ignored.

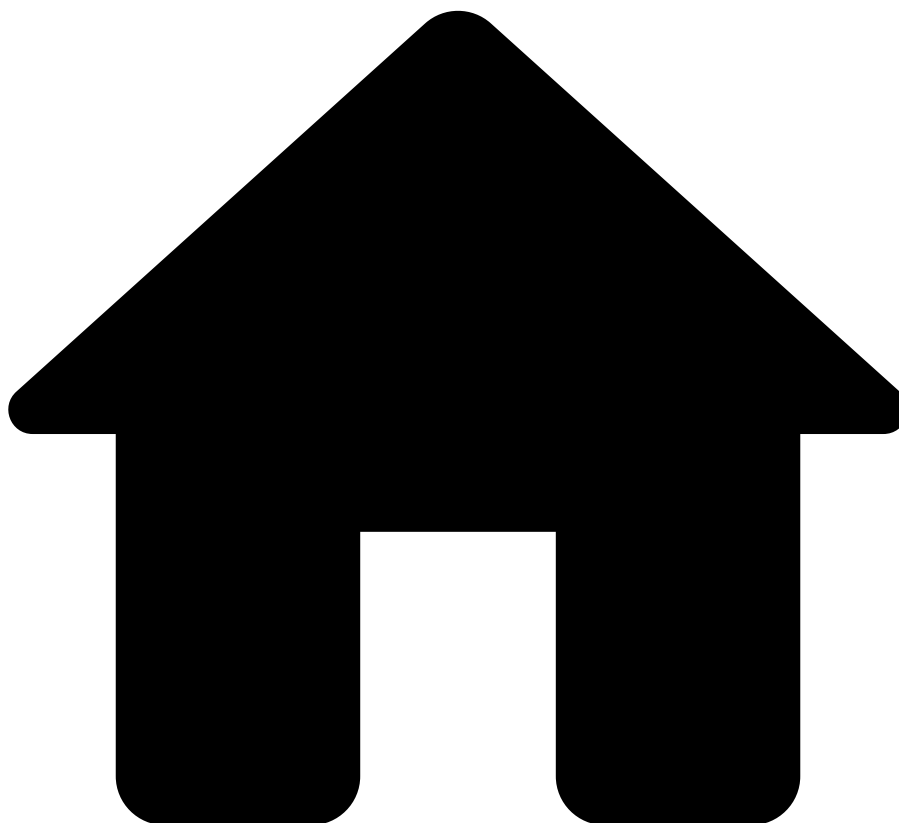
- `OS.PAN_OS`
 - No lines are ignored.
- For all other values of `os`, a line is ignored if any of the following are true.
 - The line is blank.
 - The line starts with
 - `!`,
 - `#`,
 - `;`,
 - `%`,
 - `building configuration...` (ignoring case), or
 - `current configuration :` (ignoring case).
 - The line starts with `Using` and contains `bytes`, such as `Using 2016 out of 260088 bytes!`.

See also [â](#)

Types [â](#)

- [OS](#)
- [String](#)
- [Collections](#)
- [ConfigLine](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- patternMatch

On this page

patternMatch

```
patternMatch(str: String, pattern: Pattern<T>) : T?
```

See [the data extraction guide](#).

See also [a](#)

Types [a](#)

- [String](#)